

UNIVERSIDAD DE ANTIOQUIA

Ajuste de parámetros sobre Wav2Vec 2.0 para la clasificación de la enfermedad de Alzheimer mediante señales de voz en condiciones de datos limitados.

Santiago Alvarez Zuluaga

Trabajo de grado presentado para optar al título de Ingeniero de Telecomunicaciones

Modalidad de Práctica

Trabajo de Grado

Asesor

Cristian David Rios Urrego, Magíster (MSc) en Ingeniería de Telecomunicaciones

Universidad de Antioquia

Facultad de Ingeniería

Ingeniería de Telecomunicaciones

Medellín, Antioquia, Colombia

2026

UNIVERSIDAD DE ANTIOQUIA
1803

Cita	Muñoz Zapata y Martínez Naranjo [1]
Referencia Estilo IEEE (2020)	[1] L. Muñoz Zapata y J. A. Martínez Naranjo, “Título: subtítulo”, Seleccione modalidad de grado, Seleccione pregrado, Universidad de Antioquia, Seleccione ciudad UdeA (A-Z), Seleccione año.

creative commons BY NC SA

Grupo de Investigación. Seleccione grupo de investigación UdeA (A-Z).

Seleccione centro de investigación UdeA (A-Z).

Seleccione biblioteca, CRAI o centro de documentación UdeA (A-Z)

Repositorio Institucional: http://bibliotecadigital.udea.edu.co

Universidad de Antioquia - www.udea.edu.co

El contenido de esta obra corresponde al derecho de expresión de los autores y no compromete el pensamiento institucional de la Universidad de Antioquia ni desata su responsabilidad frente a terceros. Los autores asumen la responsabilidad por los derechos de autor y conexos.

Dedicatoria

Texto de dedicatoria centrado.

Agradecimientos

Texto de agradecimientos centrado.

TABLA DE CONTENIDO

RESUMEN..... 10

ABSTRACT..... 11

I. INTRODUCCIÓN..... 12

A. *Planteamiento del problema*..... 13

B. Estado del arte..... 14

C. Justificación..... 16

II. OBJETIVOS..... 17

A. Objetivo general..... 17

B. Objetivos específicos..... 17

III. MARCO TEÓRICO..... 19

A. Wav2vec 2.0..... 19

1) Pre entrenamiento de WaV2Vec 2.0:..... 19

2)Feature encoder:..... 23

3) Encoder transformer:..... 23

a) Atención multi-cabeza...... 24

b) Suma residual...... 24

c) LayerNorm...... 24

d) Feed-forward...... 25

e) Segunda suma residual y LayerNorm...... 25

3) Etapa de clasificación:..... 25

4) Interpretabilidad por capas de representación auto-supervisadas:..... 26

a) Correlación Canonica (CCA)..... 27

b) Información Mutua (MI)..... 28

c) Localización empírica de los picos de información:.....28
B. PEFT.....29
1) Adaptadores (Adapters):.....31
2). Low-Rank Adaptation (LoRA):.....32
IV. METODOLOGÍA.....36
A. Base de datos.....37
1) Origen y tarea utilizada.....37
2) Composición y metadatos.....38
3) Preprocesamiento aplicado por los organizadores.....38
4) Partición para entrenamiento y validación.....39
B. Preprocesamiento:.....39
1) Remuestreo.....39
2) Construcción de la señal por sujeto.....39
3) Enventanado.....40
4) Selección de la duración de las ventanas.....40
C. Modelo preentrenado:.....41
1) Modelos candidatos.....41
2) Criterio de selección.....42
D. Escenarios experimentales:.....43
1) Tipos de experimento:.....43
3) Descongelamiento parcial.....44
4) Adapters de tipo bottleneck.....44
5) LoRA.....46
E. Etapa de clasificación.....47
F. Selección de modelo y evaluación.....47

V. ANÁLISIS DE RESULTADOS.....49

VI. CONCLUSIONES Y RECOMENDACIONES.....52

REFERENCIAS.....53

ANEXOS.....55

LISTA DE TABLAS

TABLA I: DISTRIBUCIÓN DE SUJETOS EN LA BASE DE DATOS ADReSS.....38
TABLA II: MODELOS WAV2VEC 2.0 EVALUADOS.....42
TABLA III: DESEMPEÑO SOBRE LOS SEGMENTOS ORIGINALES DE ADReSS.....49
TABLA IV: DESEMPEÑO DEL BASELINE POR TAMAÑO DE VENTANA.....50
TABLA V: RESULTADO DE EXPERIMENTOS PARA VENTANAS DE 6 SEGUNDOS.....52
TABLA VI: MEJORES RESULTADOS PARA 6 SEGUNDOS.....54
TABLA VII: PARAMETROS ENTRENABLES POR CONFIGURACIÓN.....56

LISTA DE FIGURAS

Fig. 1. Flujo del Preentrenamiento de Wav2Vec 2.0.....20
Fig. 2. Estructura de una capa del encoder Transformer de Wav2Vec 2.0.....24
Fig. 3.Taxonomía de métodos PEFT, con los adapters y LoRA resaltados. Adaptada de Lialin et al. [23].....30
Fig. 4. Estructura interna de un adapter bottleneck: proyección descendente, no linealidad y proyección ascendente, con conexión residual.....31
Fig. 5. Factorización de rango de la actualización de pesos ΔW en las matrices de bajo rango B y A.....34
Fig. 6: Metodología implementada.....36
Fig. 7: Adaptador Bottleneck en configuración Pfeiffer.....45
Fig. 8: Adaptador Bottleneck en configuración Houlsby.....46
Fig. 9: EXACTITUD DE MODELOS PARA VENTANAS DE 6 SEGUNDOS.....53
Fig. 10: CURVA ROC Y DIAGRAMA DE DISTRIBUCIÓN PARA EL MEJOR MODELO DE 6s.....55
Fig. 11: EXACTITUD CONTRA NUMERO DE PARAMETROS DE CADA MODELO EN VENTANAS DE 6 SEGUNDOS.....57

SIGLAS, ACRÓNIMOS Y ABREVIATURAS

Cms.	Centímetros
Esp.	Especialista
ERIC	Education Resources Information Center
IEEE	Institute of Electrical and Electronics Engineers
MP	Magistrado Ponente
MSc	Magister Scientiae
Párr.	Párrafo
PhD	Philosophiae Doctor
PBQ-SF	Personality Belief Questionnaire Short Form
PostDoc	PostDoctor
UdeA	Universidad de Antioquia

RESUMEN

La enfermedad de Alzheimer (AD) altera el habla desde etapas tempranas, lo que convierte a la voz en un biomarcador no invasivo y de bajo costo para su detección. Los modelos auto-supervisados como wav2vec 2.0 ofrecen representaciones acústicas robustas, pero su ajuste completo tiende a sobreajustar cuando los datos etiquetados son escasos. Este trabajo evalúa estrategias de ajuste fino de parámetros (PEFT) sobre wav2vec 2.0 para clasificar pacientes con AD y controles sanos a partir de habla espontánea del corpus ADReSS 2020. Se compararon tres estrategias de ajuste diferentes, descongelamiento parcial, los adaptadores bottleneck (Houlsby y Pfeiffer) y LoRA, bajo tres esquemas de ubicación: estándar, progresivo e inteligente. Un barrido previo mostró que las ventanas de hasta 4 s llevan al colapso del clasificador, por lo que los experimentos se realizaron con ventanas de 6 y 7.5 s. Con 6 s, el adaptador Houlsby ubicado en el bloque semántico alcanzó una exactitud de 83.3 % y un AUC de 0.873, frente a 66.7 % y 0.705 del modelo congelado. Con 7.5 s, LoRA inteligente obtuvo 77.1 % de exactitud (AUC de 0.818) entrenando solo el 0.026 % de los parámetros del modelo. Las configuraciones estándar no superaron el *baseline*, y en LoRA la adaptación de las matrices de valor y salida fue la más efectiva en ambas ventanas. Los resultados indican que ubicar la adaptación de forma informada importa más que el número de parámetros entrenados.

Palabras clave — E**nfermedad de Alzheimer, wav2vec 2.0, ajuste fino de parámetros, adaptadores, LoRA.**

ABSTRACT

Alzheimer's disease (AD) affects the way people speak from the early stages of the disease. Therefore, voice can be used as a low-cost and non-invasive biomarker for detecting the disease. Self-supervised models such as wav2vec 2.0 learn strong acoustic representations; however, fine-tuning the entire model with scarce labeled data may lead to overfitting. This work assesses fine-tuning strategies for wav2vec 2.0 to classify patients with AD against healthy controls using spontaneous speech from the ADReSS 2020 corpus. Three different fine-tuning strategies were compared: partial unfreezing, bottleneck adapters (Houlsby and Pfeiffer), and LoRA, under three adaptation-location schemes: Standard, Progressive, and Smart. A preliminary sweep showed that windows shorter than 4 s led to classifier collapse; therefore, the experiments were conducted using 6- and 7.5-second windows. For 6-second windows, the Houlsby adapter located in the semantic block achieved an accuracy of 83.3% and an AUC of 0.873, compared with 66.7% accuracy and an AUC of 0.705 for the original frozen model. For 7.5-second windows, Smart LoRA achieved an accuracy of 77.1% (AUC = 0.818) while training only 0.026% of the model's parameters. The Standard configuration did not outperform the baseline. Furthermore, adaptation of the value and output matrices in LoRA was the most effective across both window lengths. These results suggest that strategically locating the adaptation may be more important than the number of trainable parameters.

Keywords — **Alzheimer's disease, wav2vec, parameter fine-tuning, adapters, LoRA**

I. INTRODUCCIÓN

A nivel mundial, la demencia ha sido reconocida por la comunidad internacional como una prioridad creciente de salud pública, dada su magnitud y su impacto en los sistemas de salud, las familias y los cuidadores [1]. Colombia no es ajena a esta tendencia: entre 2015 y 2024, las muertes asociadas a enfermedades mentales y de la conducta —entre ellas el Alzheimer— se multiplicaron casi cinco veces, y solo en 2024 se registraron 4.320 muertes atribuidas específicamente a esta enfermedad, según cifras del DANE [2]. Sin embargo, a pesar de esta carga creciente, estudios recientes señalan que la demencia continúa siendo un problema de salud pública invisibilizado en el país, sin que existan a la fecha políticas públicas específicas que respondan a su magnitud [3]. Esta brecha resulta particularmente paradójica si se considera que Colombia alberga uno de los hallazgos genéticos más relevantes a nivel mundial para la investigación del Alzheimer de inicio temprano: la mutación PSEN1 E280A, conocida como «gen paisa», presente en una familia extendida de la región antioqueña y estudiada durante más de cuatro décadas como modelo único para comprender la progresión clínica de la enfermedad años antes de la aparición de los síntomas [4].

El Alzheimer es una enfermedad neurológica progresiva que, más allá del deterioro de la memoria, compromete funciones cognitivas asociadas al lenguaje; las dificultades comunicativas se cuentan entre los grupos de síntomas que con mayor frecuencia acompañan la enfermedad [5]. El diagnóstico del Alzheimer se ha apoyado tradicionalmente en la evaluación clínica y la neuroimagen estructural, complementadas en las últimas dos décadas por pruebas de biomarcadores en líquido cefalorraquídeo (LCR) y tomografía por emisión de positrones (PET); sin embargo, estas técnicas presentan baja sensibilidad en etapas tempranas y un acceso limitado debido a su costo elevado y su naturaleza invasiva [6]. Teniendo en cuenta este panorama, resulta urgente explorar alternativas de detección que superen las limitaciones de los métodos diagnósticos tradicionales. Muchas evaluaciones neurológicas, aunque clínicamente valiosas, no están diseñadas para ser realizadas de forma masiva y constante atendiendo a la magnitud que el problema exige, retrasando la intervención terapéutica y empeorando el pronóstico de los pacientes [5]. Frente a estas barreras, los biomarcadores han cobrado relevancia como una estrategia complementaria capaz de ofrecer diagnósticos más accesibles y económicos [6]. En

este contexto, el análisis de habla constituye un procedimiento simple, económico y no invasivo, con potencial para la detección del Alzheimer incluso en etapas preclínicas [5]. No obstante, el desarrollo de sistemas automáticos de clasificación basados en habla conlleva una limitación estructural reconocida en la literatura: el tamaño reducido de las muestras disponibles que responde a una dificultad en la recolección de datos de habla en poblaciones con deterioro cognitivo [5].

Los modelos de aprendizaje autosupervisado han revolucionado la manera en que se generan representaciones de habla, permitiendo capturar estructuras acústicas y lingüísticas ricas a partir de grandes volúmenes de audio sin necesidad de transcripciones o etiquetas manuales [7]. Sin embargo, este paradigma no elimina la dependencia de datos: el pre entrenamiento se apoya en corpus masivos no etiquetados, y el ajuste posterior a una tarea específica (el fine-tuning) asume también la disponibilidad de conjuntos de datos etiquetados suficientemente amplios para ajustar el gran número de parámetros que un modelo de este tamaño requiere.

A. Planteamiento del problema

Esta suposición representa un gran problema al trasladar estos métodos al dominio clínico de las enfermedades neurodegenerativas: las bases de datos de habla con etiquetas de Alzheimer disponibles públicamente son, por naturaleza, reducidos. ADReSS reúne apenas 156 hablantes [8], esta cantidad de muestras es órdenes de magnitud menores que los corpus empleados para el pre entrenamiento de Wav2Vec 2.0.

Frente a esta tensión, han surgido estrategias de ajuste eficiente de parámetros, conocidas en inglés como Parameter-Efficient Fine-Tuning (PEFT) diseñadas justamente para adaptar modelos de gran tamaño con pocos datos; sin embargo, estas estrategias no se han aplicado a la clasificación de Alzheimer frente a controles sanos a partir de Wav2Vec 2.0. Es en esta ausencia donde se sitúa el aporte metodológico central de la presente propuesta. ¿Cómo construir un clasificador robusto, capaz de distinguir entre hablantes con Alzheimer y controles sanos, en condiciones de datos limitados?

B. Estado del arte

El desafío ADReSS [8] estableció un punto de referencia común para la detección automática del Alzheimer a partir del habla: 156 hablantes balanceados por clase, edad y género, grabaciones con reducción de ruido y normalización de volumen, y una partición fija de entrenamiento y prueba. Su línea base acústica, construida con descriptores diseñados manualmente, alcanzó una exactitud de 62,5 %, lo que evidenció que las características acústicas tradicionales capturan solo parcialmente la enfermedad. Haider et al. [9] evaluaron conjuntos estandarizados como eGeMAPS y ComParE en habla espontánea de pacientes con Alzheimer. Encontraron que estos conjuntos capturan parte de los cambios vocales asociados a la enfermedad, pero su alcance queda restringido a los descriptores definidos a priori a partir del conocimiento de un comité de expertos. Esta restricción motivó el uso del aprendizaje profundo, que busca aprender la representación directamente de la señal, aunque a costa de enfrentar el principal obstáculo de estos modelos: el tamaño reducido de los corpus clínicos. Rios-Urrego et al. [10] abordaron este obstáculo en la detección de la enfermedad de Parkinson entrenando redes convolucionales con habla de pacientes en un idioma y usaron los pesos aprendidos como punto de partida para otro idioma, congelando parte de sus capas y reentrenando las demás. De esta forma mostraron que reutilizar el conocimiento previamente aprendido en una red es una vía para compensar la escasez de datos. Sin embargo el modelo original se entrenó con habla de pacientes etiquetada por diagnóstico. La transferencia de los pesos trasladaba la dependencia de etiquetas clínicas de un corpus a otro sin eliminarla. Se requería, por tanto, aprender representaciones del habla sin etiquetas, de modo que los datos clínicos etiquetados solo fueran necesarios en la etapa final de adaptación.

Wav2Vec 2.0 [7] eliminó la dependencia de etiquetas en la etapa de preentrenamiento. Con apenas diez minutos de datos etiquetados, el modelo alcanzó un reconocimiento de habla competitivo. No obstante, fue diseñado y evaluado para reconocimiento de habla, y no era claro qué información aporta cada una de sus capas a otras tareas. Pasad et al. [11] respondieron esa pregunta midiendo, capa por capa, el contenido acústico, fonético y léxico del modelo. Encontraron una trayectoria similar a la de un autoencoder: las capas intermedias concentran la mayor información fonética, las finales vuelven a parecerse a la entrada y el fine-tuning modifica principalmente las capas superiores. Su análisis, sin embargo, se limitó a habla sana.

La adaptación a pequeñas bases de datos de habla patologica enfrenta un riesgo documentado. Berisha et al. [12] analizaron estudios publicados de clasificación clínica a partir del habla y mostraron que la exactitud reportada disminuye a medida que aumenta el tamaño de la muestra. Si los datos provienen de la misma distribución y describen el mismo fenómeno, un modelo confiable deberia mantener o mejorar su desempeño al aumentar la cantidad de datos. Liu et al. [13] expusieron un modelo ajustado únicamente con los segmentos de silencio del Pitt corpus alcanzando 98,9 % de exactitud, cifra que cayó a 56,7 % en ADReSS con grabaciones normalizadas. Xiong et al. [14] mostraron que ajustar un modelo fundacional de habla con un corpus clínico de tamaño limitado conduce a sobreajuste. Al ajustar el Transformer de Wav2Vec 2.0, la pérdida sobre hablantes no vistos aumentó a lo largo del entrenamiento, pese al uso de regularización. Los autores lo atribuyen al desequilibrio entre la gran capacidad del modelo y la escasez de datos de la tarea: el modelo termina capturando patrones de los datos de entrenamiento ajenos a la patología en lugar de las características relevantes, y generaliza mal.

Esto sugiere que, con pocos datos, adaptar todo el modelo es arriesgado y que conviene entrenar solo una fracción de sus parámetros, una idea desarrollada originalmente en el procesamiento de lenguaje natural. ULMFiT [15] introdujo el descongelamiento gradual de capas, de las superiores a las inferiores, con tasas de aprendizaje distintas por capa. Con 100 ejemplos etiquetados igualó a modelos entrenados desde cero con cien veces más datos, pero al final del proceso ajusta todos los pesos y exige una copia completa del modelo por tarea. Houlsby et al. [16] evitaron esa copia con adaptadores tipo bottleneck: módulos pequeños con conexiones residuales insertados en cada capa, los únicos que se entrenan. Quedaron a 0,4 % del fine-tuning completo añadiendo solo 3,6 % de parámetros por tarea, aunque los módulos adicionales incrementan el cómputo en inferencia. LoRA [17] eliminó ese costo: mantiene fijas las matrices de pesos W y aprende una actualización de bajo rango, $W' = W + BA$, que se integra a los pesos sin añadir latencia y redujo miles de veces los parámetros entrenables con un desempeño comparable. Estas técnicas, no obstante, fueron validadas en texto. Chen et al. [18] las trasladaron a modelos de habla autosupervisados y alcanzaron un desempeño comparable al fine-tuning completo reduciendo en más de un 90 % los parámetros entrenables, aunque en tareas de habla general, sin datos clínicos.

Purohit et al. [19] dieron ese paso en la detección de la enfermedad de Parkinson: compararon la selección de capas congeladas, el fine-tuning completo y LoRA sobre modelos fundacionales de habla. Seleccionar la décima capa congelada de Wav2Vec 2.0 (83,54 %) superó al fine-tuning completo (80,90 %), y LoRA aplicado a Whisper alcanzó 85,00 %, por encima de su contraparte ajustada por completo (83,53 %). Su estudio, sin embargo, se limita a la enfermedad de Parkinson, un trastorno principalmente motor, y a LoRA, sin evaluar adaptadores ni descongelamiento parcial. Ninguna de estas estrategias se ha evaluado sobre Wav2Vec 2.0 para la clasificación de Alzheimer frente a controles sanos, una enfermedad cuya huella en el habla es principalmente cognitiva y lingüística. Ese es el vacío que aborda la presente tesis.

C. Justificación

Frente a esta escasez de datos, en este trabajo se propone aplicar y comparar diferentes estrategias de adaptación sobre Wav2Vec 2.0 para la tarea de clasificación binaria entre hablantes con Alzheimer y controles sanos: adaptadores tipo bottleneck, que insertan módulos secuenciales de baja dimensionalidad entre las capas del modelo [20]; adaptaciones de bajo rango, conocidas por sus siglas en inglés LoRA (Low Rank Adaptation), que introduce una corrección de bajo rango sobre las matrices de pesos existentes sin modificarlas directamente [21]; y el descongelamiento parcial de las últimas n capas del modelo, como forma más directa de ajuste. Estas tres estrategias serán evaluadas frente a un mismo punto de referencia fijo: Wav2Vec 2.0 sin ningún ajuste de sus pesos. A partir de este diseño experimental, la presente tesis plantea como hipótesis central que el ajuste del modelo, mediante cualquiera de las estrategias mencionadas, permite obtener un desempeño de clasificación superior al del modelo congelado, incluso en condiciones de datos etiquetados limitados.

II. OBJETIVOS

A. Objetivo general

Desarrollar una estrategia de ajuste de parámetros sobre modelos fundacionales auto-supervisados, como Wav2Vec 2.0, para la clasificación de pacientes con enfermedad de Alzheimer frente a controles sanos mediante señales de voz en condiciones de datos limitados.

B. Objetivos específicos

- Establecer una línea base de clasificación de pacientes con enfermedad de Alzheimer y sujetos de control mediante el uso de representaciones extraídas por Wav2Vec 2.0 sin ajuste de parámetros.
- Identificar estrategias de ajuste de parámetros aplicables a la arquitectura de Wav2Vec 2.0 para tareas de clasificación binaria en escenarios con datos limitados.
- Implementar diferentes estrategias de ajuste de parámetros sobre el modelo fundacional Wav2Vec 2.0, con el fin de adaptarlo a la tarea de clasificación binariaentre pacientes con la enfermedad de Alzheimer y sujetos de control.
- Evaluar y comparar el desempeño de clasificación, el tiempo de entrenamiento, el tiempo de inferencia y el número de parámetros entrenables entre el enfoque del modelo base y las estrategias de ajuste implementadas.

III. MARCO TEÓRICO

A. Wav2vec 2.0

Wav2Vec 2.0 [7] es un modelo de aprendizaje autosupervisado que aprende representaciones de habla directamente del audio crudo sin necesidad de transcripciones. Para comprender su arquitectura, es importante distinguir dos etapas: el preentrenamiento autosupervisado y el uso posterior del modelo preentrenado. Durante el preentrenamiento, el modelo aprende sus representaciones de forma autosupervisada; mediante una arquitectura que incluye, dos codebooks y dos funciones de pérdida (contrastiva y de diversidad). De manera general al utilizar este tipo de modelos se parte del modelo ya preentrenado y nunca se reconstruye ni ejecuta el cuantizador, los codebooks o las pérdidas de preentrenamiento.

1) Pre entrenamiento de WaV2Vec 2.0:

El *feature encoder* transforma la señal cruda en una secuencia de vectores latentes continuos $z_1, \ldots . z_T$, uno por cada fotograma de 25 ms. Esta secuencia z alimenta en paralelo dos ramas independientes durante el *forward*. En la primera, antes de ingresar al *Transformer*, se enmascaran tramos contiguos: se muestrea una proporción $p \approx 65$ de los pasos como índices de inicio, y desde cada uno se cubren M=10 pasos consecutivos con un único vector entrenable compartido (el *mask embedding*). Por el solapamiento entre tramos, el resultado es que cerca del 49% de la secuencia queda enmascarada. Esta estrategia de bloques (en vez de fotogramas aislados) sigue una lógica similar a *SpecAugment [22]*: los fotogramas vecinos están altamente correlacionados, de modo que enmascarar uno solo permitiría reconstruirlo por simple interpolación, mientras que cubrir un tramo completo obliga al modelo a recurrir a contexto de largo alcance.

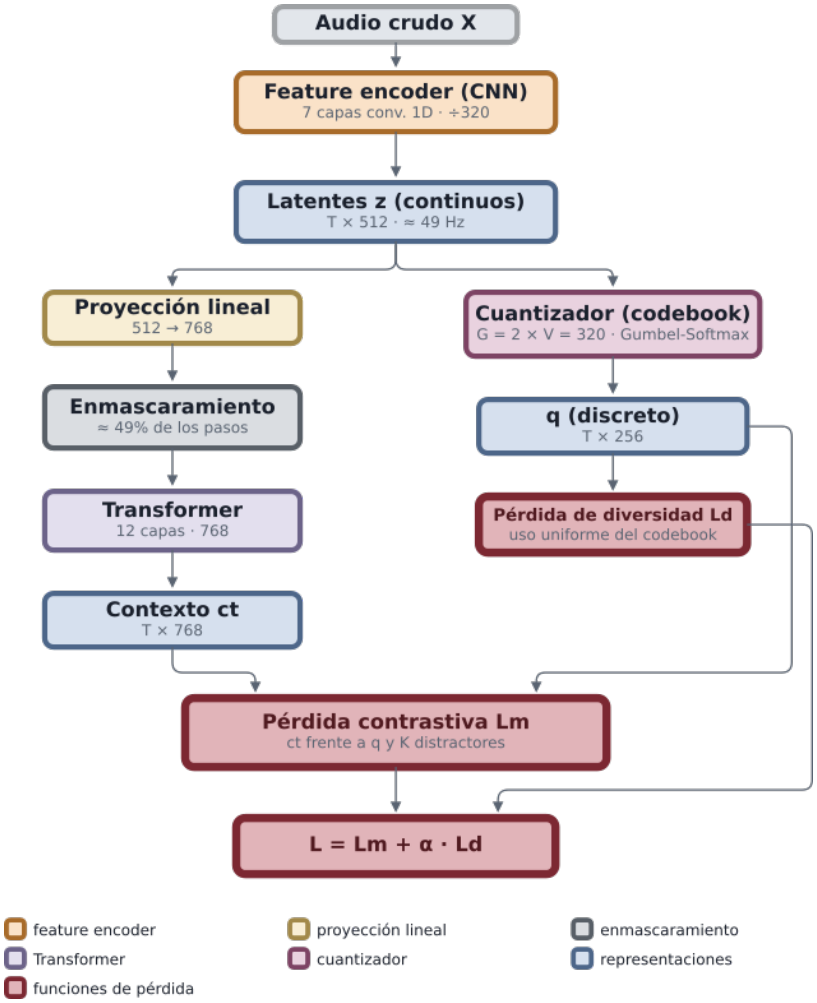


Fig. 1. Flujo del Preentrenamiento de Wav2Vec 2.0

La secuencia z parcialmente enmascarada entra al Transformer, que produce una representación contextualizada c_t para cada posición mediante auto-atención: cada posición, actuando como *query* Q, calcula su similitud contra las *keys* K de todas las posiciones mediante el producto punto $Q \cdot K$; luego se normaliza esos puntajes con una función *softmax*, y construye su salida como una suma ponderada de los *values* Vde las posiciones a las que atiende. En las posiciones enmascaradas, dado que el *mask embedding* no contiene información real del fotograma original, c_t se reconstruye enteramente a partir del contexto (las posiciones vecinas no enmascaradas) y no de su propia entrada.

En la segunda rama, la misma secuencia z alimenta un cuantizador que discretiza cada vector en una representación de un conjunto finito de unidades aprendidas, mediante dos codebooks (G=2 grupos), cada uno es una matriz de V=320 filas por d/G columnas, (siendo d la dimensión del vector que resulta de concatenar las G filas seleccionadas) inicializadas aleatoriamente y ajustadas por la propagación hacia atrás como cualquier otro peso del modelo. Para cuantizar un fotograma, una capa lineal proyecta z a un conjunto de logits $l \in \mathbb{R}^{G \times V}$; un mecanismo de *Gumbel-softmax* selecciona un índice por grupo; se extraen las filas correspondientes de cada *codebook* y se concatenan en un vector de dimensión d; y una transformación lineal final produce el vector cuantizado q. Esta estructura de *product quantization* permite representar hasta V^2=102.400 combinaciones distintas almacenando únicamente $G \times V = 640$ vectores, una ganancia de eficiencia frente a un *codebook* único que requeriría almacenar esas 102.400 combinaciones de forma independiente.

La selección discreta de filas (una operación *argmax*, no diferenciable) se entrena mediante *straight-through estimator* combinado con ruido de *Gumbel*. Sobre cada logit $l_{g,v}$ se suma un ruido n_v muestreado de una distribución de *Gumbel*; la propiedad conocida como *Gumbel-max trick* garantiza que $argmax(l+n)$ equivale a muestrear una fila con probabilidad $softmax(l)$, introduciendo la exploración necesaria durante el entrenamiento. Para permitir la propagación del gradiente se define:

$$p_{g,v} = \frac{exp(l_{g,v}+n_v)/\tau}{\sum_{k=1}^{V} exp(l_{g,k}+n_k/\tau)} \quad (1)$$

En el *forward* se toma el *argmax* de esta distribución (selección dura, tipo *one-hot*, que ignora τ); en el *backward* se emplea el gradiente del *softmax* completo, suave y diferenciable, con una temperatura τ positiva que decrece gradualmente durante el entrenamiento (de 2 a 0.5, sin llegar nunca a 0).

El modelo se entrena mediante dos pérdidas combinadas. La pérdida contrastiva $\mathcal{L}_m$ evalúa, en cada posición enmascarada, si c_t permite distinguir su versión cuantizada verdadera q_t

frente a un conjunto de distractores Q_t (otros q muestreados de otras posiciones enmascaradas del mismo audio), mediante similitud coseno escalada por una temperatura k :

$$\mathcal{L}_m = -\log \frac{\exp\left(sim(\mathbf{c}_t, \mathbf{q}_t)/\kappa\right)}{\sum_{\tilde{\mathbf{q}} \sim \mathbf{Q}_t} \exp\left(sim(\mathbf{c}_t, \tilde{\mathbf{q}})/\kappa\right)} \quad (2)$$

Una pérdida de diversidad $\mathcal{L}_d$ evita que el cuantizador colapse en el uso repetido de solo unas pocas filas de cada *codebook*, maximizando la entropía del uso promedio a lo largo del *batch*:

$$\mathcal{L}_d = \frac{1}{GV} \sum_{g=1}^{G} -H(\bar{p}_g) = \frac{1}{GV} \sum_{g=1}^{G} \sum_{v=1}^{V} \bar{p}_{g,v} \log \bar{p}_{g,v} \quad (3)$$

Donde $\bar{p}_g$ es la probabilidad de selección de cada fila, promediada sobre los fotogramas del *batch* (con un *softmax* plano, sin ruido ni temperatura). El objetivo total combina ambas pérdidas $\mathcal{L} = \mathcal{L}_m + \alpha \mathcal{L}_d$, donde α regula el balance entre utilidad y diversidad de las unidades seleccionadas.

Terminado el preentrenamiento, el cuantizador, los dos codebooks y ambas pérdidas se descartan. Lo único que se conserva y se distribuye en el modelo público es el *feature encoder* y el *encoder Transformer*, más una etapa de clasificación añadida que dependera del fenomeno en el que estemos interesados analizar.

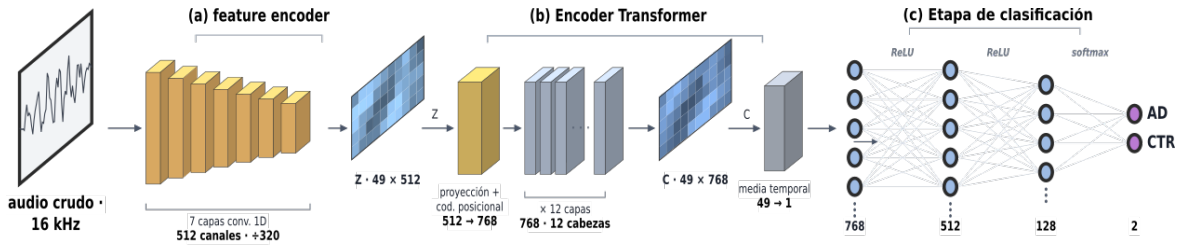


Fig 2: Arquitectura de Wav2Vec 2.0 para clasificación: (a) feature encoder, (b) encoder Transformer y (c) Etapa de clasificación.

2)Feature encoder:

El feature encoder es la primera fase de la arquitectura de Wav2Vec 2.0 y opera directamente sobre el audio crudo, sin ningún paso intermedio de extracción de características tipo espectrograma. Es la misma etapa que produce la secuencia de vectores latentes z descrita en la sección anterior: siete bloques convolucionales 1D, cada uno con 512 canales, con anchos de kernel (10, 3, 3, 3, 3, 2, 2) y strides (5, 2, 2, 2, 2, 2, 2) [7].

El producto de estos siete strides (5×2×2×2×2×2×2 = 320) determina el factor de *downsampling* temporal total de la capa: por cada 320 muestras de audio crudo a la entrada, el feature encoder produce un único vector de z a la salida. Para audio muestreado a 16 kHz, esto se traduce en una tasa de salida aproximada de 49 Hz, es decir, un vector cada ~20 ms, de modo que cada posición de z resume una ventana corta y solapada de la señal cruda, sin ningún tipo de contexto, el cual se incorpora más adelante, en el encoder Transformer.

3) Encoder transformer:

La secuencia z, tras el feature encoder, se proyecta de 512 a 768 dimensiones y se le suma una codificación posicional relativa, implementada como una capa convolucional (kernel de tamaño 128, 16 grupos) en lugar de los embeddings posicionales sinusoidales fijos habituales en el Transformer original. Esto es necesario porque la atención, a diferencia de una convolución o una recurrencia, no distingue por sí sola el orden de los elementos de la secuencia: sin esta señal, el modelo trataría a z como un conjunto no ordenado de vectores.

Cada una de las 12 capas del encoder repite el mismo bloque (Fig. 3), compuesto por los siguientes elementos:

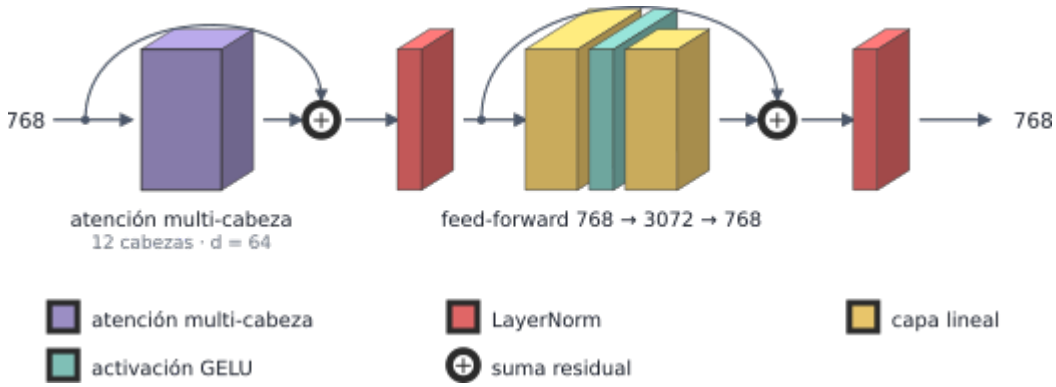


Fig. 2. Estructura de una capa del encoder Transformer de Wav2Vec 2.0.

a) Atención multi-cabeza.

Es el mecanismo que introduce contexto: cada posición, actuando como *query*, calcula su similitud contra las *keys* de todas las demás posiciones de la secuencia mediante el producto punto escalado; esos puntajes se normalizan con una función *Softmax* y la salida de esa posición se reconstruye como una suma ponderada de los *values* de todas las posiciones, dando más peso a aquellas con las que tiene mayor similitud. Que sea "multi-cabeza" (12 cabezas de d=64 cada una, sumando 768) significa que esta operación se repite en paralelo 12 veces, cada una sobre una proyección distinta del mismo vector; esto permite que cada cabeza se especialice en capturar un tipo distinto de relación entre diferentes posiciones.

b) Suma residual.

La salida de las cabezas de atención no reemplaza la entrada original del bloque, con el fin de evitat que la información original del fotograma se pierda al pasar por la atención y facilitar la propagación del gradiente durante el entrenamiento cuando se apilan muchas capas el contenido original que entra al *encoder s*e suma con la salida contextualizada. Sin esta suma, el gradiente tendería a degradarse a medida que la señal atraviesa las 12 capas.

c) LayerNorm.

Normaliza el vector resultante (media 0, varianza 1 a lo largo de la dimensión de características) antes de continuar. Esto estabiliza la magnitud de las activaciones capa tras capa, evitando que crezcan o se degraden de forma descontrolada conforme la señal avanza por las 12 capas.

d) Feed-forward.

Es una red de dos capas lineales aplicada de forma independiente a cada posición de la secuencia: primero expande la dimensión de 768 a 3072, aplica una no linealidad GELU, y luego proyecta de vuelta a 768. Mientras que la atención decide de qué otras posiciones tomar información, la feed-forward transforma el contenido de cada posición de forma no lineal, añadiendo capacidad de representación que la atención por sí sola no provee.

e) Segunda suma residual y LayerNorm.

Se repite el mismo patrón: la salida de la feed-forward se suma a su entrada (por la misma razón que la primera suma residual) y el resultado se normaliza, cerrando el bloque.

Apilando 12 de estos bloques se obtiene, para cada fotograma de la secuencia, una representación contextualizada C (49×768) que a diferencia de z, que solo resume una ventana corta de audio crudo, ya incorpora información de toda la secuencia gracias a las 12 rondas sucesivas de atención.

3) Etapa de clasificación:

En el paper original, Wav2Vec2.0 se evalúa fundamentalmente como modelo base para reconocimiento automático de habla (ASR): la tarea de transcribir una señal de audio a su secuencia de texto correspondiente [7]. Para esto, la cabecera que se coloca sobre el Transformer preentrenado es una única capa lineal, inicializada aleatoriamente, que proyecta cada vector de la secuencia contextualizada C a un conjunto de logits sobre el vocabulario de la tarea; en el *corpus* LibriSpeech ese vocabulario son 29 tokens de caracteres más un token de separación de palabra [x] , y en TIMIT (reconocimiento de fonemas) son 39 clases fonéticas [x]. Esta etapa conserva la longitud temporal de C (produce una predicción por cada fotograma, no una única predicción por audio) porque el problema que resuelve es de secuencia a secuencia sin un alineamiento conocido entre fotogramas de entrada y símbolos de salida; por eso se entrena con la pérdida CTC (Connectionist Temporal Classification), que evita necesitar ese alineamiento sumando sobre todas las segmentaciones posibles compatibles con la secuencia a encontrar.

Adaptar Wav2Vec 2.0 a una tarea de clasificación implica reutilizar el modelo preentrenado, descartando el cuantizador (empleado solo en preentrenamiento) y reemplazando la etpa de CTC por una nueva dado que C tiene longitud variable (depende de la duración del audio) y una tarea de clasificación requiere una única predicción por audio. Debido a esto es necesario colapsar C a un vector de dimensión fija antes de clasificar. Esto se resuelve con una operación de pooling temporal (las variantes más comunes en la literatura son mean pooling, max pooling o pooling ponderado por atención) seguida de una o más capas que proyectan ese vector fijo a K clases. Al desaparecer el problema de alineamiento (ya no hay una secuencia de salida que alinear con la de entrada), la pérdida cambia de la formulación contrastiva de preentrenamiento a una entropía cruzada estándar:

$$\mathcal{L}_{CE} = -\sum_{c} y_c \log(\hat{y}_c) \tag{4}$$

Donde y_c es la etiqueta verdadera y $\hat{y}_c$ la probabilidad predicha para la clase c.

4) Interpretabilidad por capas de representación auto-supervisadas:

Una red neuronal profunda como wav2vec 2.0 está compuesta por una secuencia de capas apiladas, cada una de las cuales transforma la representación recibida de la capa anterior en una nueva representación intermedia. Si bien el desempeño de estos modelos está bien documentado en diferentes tareas, esto no implica que se conozca qué tipo de información codifica cada capa individualmente, ni como esa información varía a lo largo de la profundidad de la red.

Existen dos aproximaciones generales para responder esta pregunta. La primera consiste en entrenar un clasificador auxiliar ("sonda" o *probe*) sobre la representación de cada capa, y evaluar su desempeño en una tarea específica (p. ej., clasificación de fonemas) [11]. El problema de este enfoque es que introduce una incertidumbre adicional: si el clasificador obtiene un desempeño bajo, no es posible determinar si esto se debe a que la capa efectivamente no codifica la información buscada, o a decisiones de diseño del propio clasificador (arquitectura, hiperparámetros, cantidad de datos de entrenamiento). Pasad et al (2021) [11] optan en su lugar por un segundo tipo de análisis: medidas estadísticas directas que comparan la representación de

una capa contra una variable externa de interés sin necesidad de entrenar ningún modelo adicional.

a) Correlación Canonica (CCA)

El Análisis de Correlación Canónica es una técnica estadística que cuantifica la relación entre dos representaciones. Dado dos vectores $\mathbf{x} \in \mathbb{R}^{\mathbf{d1}}$ (por ejemplo, la salida de una capa del transformer) e $\mathbf{y} \in \mathbb{R}^{\mathbf{d2}}$ (por ejemplo, una etiqueta lingüistica externa, como la identidad fonetica), CCA busca un par de direcciones de proyección *v* y *w* que maximizan la correlación entre las proyecciones lineales resultantes

$$v_1, w_1 = argmax_{v,w} corr(\mathbf{v^T x}, \mathbf{w^T y}) \text{ (x)}$$

El procedimiento se repite iterativamente para encontrar hasta $K = min(d1, d2)$ pares de direcciones adicionales, cada una sujeta a la restricción de no ser redundante con las anteriores, obteniendo así un conjunto de correlaciones $\rho_1, \rho_2, ..., \rho_k$.

Pasad et al. (2021) emplean específicamente projection-weighted CCA (PWCCA; Morcos et al., 2018), una variante que pondera cada correlación ρ_i de forma proporcional a la relevancia real de esa dirección dentro del espacio original, en lugar de promediarlas con igual peso.

$$PWCCA(X, Y) = \frac{\sum_i \alpha_i \cdot \rho_i}{\sum_i \alpha_i} \text{ (x)}$$

Esta ponderación es necesaria porque no todas las direcciones encontradas son igual de confiables: al comparar espacios de alta dimensionalidad con relativamente pocas muestras de datos, algunas de las correlaciones encontradas por CCA reflejan patrones reales, mientras que otras son producto del azar estadístico, por lo tanto, promediarlas con el mismo peso que las direcciones realmente informativas sesgaría la medida final.

b) Información Mutua (MI)

Como complemento a CCA, Pasad et al. (2021) [12] emplean información mutua para capturar dependencias estadísticas de cualquier tipo (no únicamente lineales) entre la representación de una capa y una etiqueta discreta externa (identidad fonética o de palabra).

Dadas dos variables discretas X e Y, la información mutua se define como:

$$I(X;Y) = \sum_{x} \sum_{y} p(x,y) \log \left(\frac{p(x,y)}{p(x)\, p(y)} \right) \quad \text{(x)}$$

Esta cantidad es nula si y solo si X e Y son estadísticamente independientes, y aumenta en la medida en que conocer una variable reduce la incertidumbre sobre la otra. Dado que las representaciones de una capa son vectores continuos y $I(X;Y)$ requiere variables discretas, los autores discretizan cada representación mediante agrupamiento no supervisado (*k-means*), asignando a cada instancia el identificador del clúster más cercano; esta etiqueta discreta de clúster reemplaza al vector continuo original en el cálculo de $I(X;Y)$.

c) Localización empírica de los picos de información:

Los autores identifican, para wav2vec2-Base (12 capas), una progresión jerárquica de tipo acústico-lingüístico:

- Capas 5-7 — contenido fonético. Tanto la información mutua entre la representación de la capa y la etiqueta real de fonema (MI-phone) como la similitud CCA contra *acoustically grounded word embeddings* (AGWE —embeddings entrenados para capturar específicamente similitud fonético-acústica) alcanzan su punto máximo en este rango.

- Capas 7-8 — contenido semántico. La similitud CCA entre la representación de la capa y embeddings de significado de palabra (GloVe) alcanza su máximo en un rango intermedio.
- Capas 9-11 — fase reconstructiva. Tanto MI-phone como MI-word decrecen fuertemente, lo que indica que el modelo deja de priorizar la información lingüística y empieza a reconstruir características cercanas a la entrada original.

Esta partición fonético/semántico/reconstructivo es el fundamento directo de la variante Inteligente, aplicada de forma consistente en los tres escenarios PEFT de esta tesis: descongelamiento parcial por bloques, adapters y LoRA usan la misma división en Bloque A (fonético), Bloque B (semántico) y Bloque C (reconstructivo) para decidir dónde intervenir, en comparación con las configuraciones de referencia agnósticas a la interpretabilidad de las salidas del modelo.

B. PEFT

La forma habitual de adaptar un modelo preentrenado a una tarea específica es el *fine-tuning* completo: se parte de los pesos aprendidos en el preentrenamiento y se continúa entrenando la totalidad de ellos con los datos etiquetados de la nueva tarea. Si Φ_0 son los pesos preentrenados, el fine-tuning completo aprende para cada tarea una actualización $\Delta\Phi$ del mismo tamaño que Φ_0 ; en consecuencia, cada tarea requiere almacenar una copia completa del modelo [21]. Adicionalmente, cuando tenemos un conjunto de datos pequeño para la tarea objetivo, ajustar la red en su totalidad puede resultar subóptimo [20], y reentrenar los pesos compartidos expone al modelo a olvidar las representaciones aprendidas previamente, fenómeno conocido como olvido catastrófico [20].

El ajuste fino eficiente en parámetros (PEFT, por sus siglas en inglés) aborda estas limitaciones manteniendo congelados los pesos preentrenados y entrenando únicamente un conjunto pequeño de parámetros nuevos. Houlsby et al [20]. lo formalizan de la siguiente manera: Dada una red $\varphi_w(x)$ con parametros preentrenados w, se define una nueva función $\psi_{w,v}(x)$ donde w se copía del preentrenamiento y permanece fijo mientras v es un conjunto de parámetros nuevos con

$|v| \ll |w|$. Los parámetros iniciales v_0 se eligen de modo que la nueva función reproduzca la original:

$$\psi_{w,v_0}(x) \approx \varphi_w(x) \quad (6)$$

De forma tal, que durante el entrenamiento solo se ajusta v. Esta condición de inicialización garantiza que el entrenamiento arranque exactamente desde el comportamiento del modelo preentrenado y se aleje de él de forma gradual desencadenando en dos atributos principales: el modelo resultante es compacto porque cada tarea añade solo unos pocos parámetros y adicionalmente es extensible debido a que se pueden incorporar tareas nuevas sin alterar los pesos compartidos con las tareas o dimensiones previamente entrenadas.

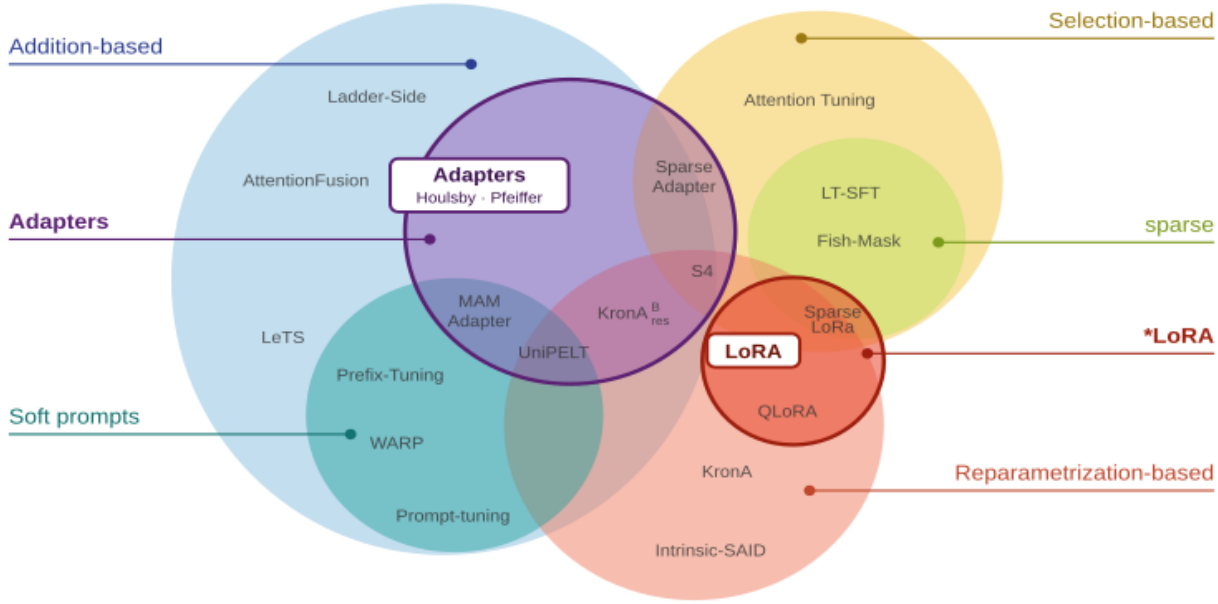


Fig. 3.Taxonomía de métodos PEFT, con los adapters y LoRA resaltados. Adaptada de Lialin et al. [23].

Las técnicas PEFT pueden clasificarse en tres clases principales según sea su enfoque [23]: los métodos aditivos, que aumentan el modelo preentrenado con parámetros o capas nuevas y entrenan únicamente esos elementos; los métodos selectivos, que ajustan un subconjunto de los

parámetros ya existentes del modelo, como las últimas capas de la red o únicamente sus sesgos; y los métodos de reparametrización, que aprovechan representaciones de bajo rango para expresar la actualización de los pesos con un número reducido de parámetros entrenables. A estas se suman los métodos híbridos, que combinan ideas de varias clases. Dentro de los métodos aditivos, a su vez, se distinguen dos grandes subgrupos: los métodos tipo adapter, que insertan pequeñas redes totalmente conectadas después de las sub-capas del Transformer, y los soft prompts, que ajustan mediante gradiente una parte de los *embeddings* de entrada del modelo. Estas técnicas buscan reducir el consumo de memoria, mejorar la eficiencia de almacenamiento o aportar modularidad, y difieren entre sí en la cantidad de parámetros entrenables y el costo de entrenamiento e inferencia [23].

1) Adaptadores (Adapters):

Un adapter es un módulo pequeño de parámetros nuevos que se inserta dentro de las capas de una red preentrenada cuyos pesos originales permanecen congelados; durante el entrenamiento solo se ajustan los parámetros del adapter [20]. A este modulo se le atribuyen dos características principales: un número reducido de parámetros en comparación con las capas de la red original, lo que hace que el tamaño total del modelo crezca lentamente a medida que se añaden tareas, y una inicialización cercana a la identidad, necesaria para que el entrenamiento sea estable.

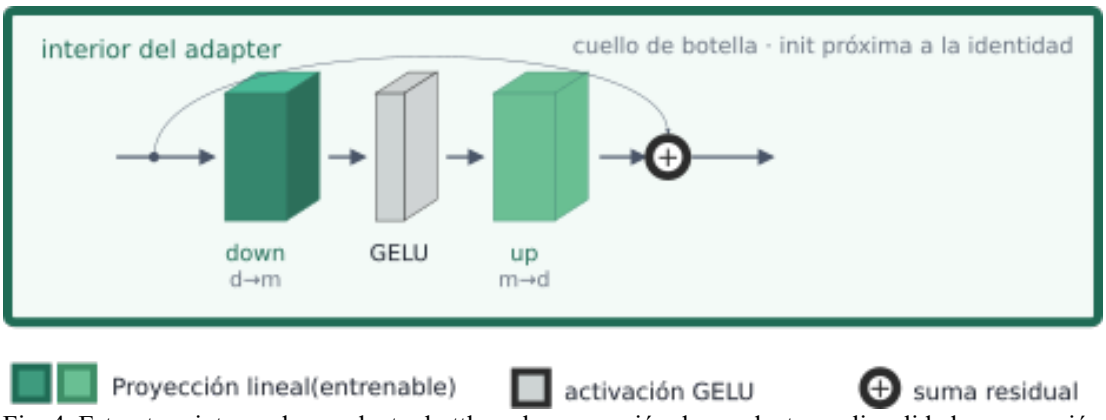


Fig. 4. Estructura interna de un adapter bottleneck: proyección descendente, no linealidad y proyección ascendente, con conexión residual.

Para limitar el número de parámetros, el adapter adopta una arquitectura de cuello de botella (Fig. 5). Dada una representación de entrada $\mathrm{h} \in \mathbb{R}^d$, una primera capa lineal (FF Down) la proyecta a una dimensión menor m, Sobre esta se aplica una no linealidad f, y una segunda capa lineal (FF Up) la proyecta de vuelta a las d dimensiones originales. El módulo incluye además una conexión residual interna que suma la entrada a esa transformación:

$$\mathrm{Adapter}(h) = h + W_{up}\, f(W_{down}\, h + b_{down}) + b_{up} \quad (7)$$

Donde $\mathrm{W}_{down} \in \mathbb{R}^{m \times d}$ y $b_{down} \in \mathbb{R}^m$ son los parámetros de FF Down, y $\mathrm{W}_{up} \in \mathbb{R}^{d \times m}$ y $b_{up} \in \mathbb{R}^d$ los de FF Up. El número total de parámetros que añade cada adapter, incluyendo los sesgos es de $2md + d + m$.

Al fijar $\mathrm{m} \ll d$, este número se mantiene pequeño; en la práctica, se utiliza entre el 0.5 y el 8% de los parámetros del modelo original [20] . Esta matriz se expresa mediante el factor de reducción r, que indica cuántas veces se reduce la dimensión oculta en el cuello de botella [20]: $r = \frac{d}{m}$.
Por ejemplo, para una dimensión oculta $\mathrm{d} = 768$, un factor $\mathrm{r} = 64$ produce un cuello de botella de $\mathrm{m} = 12$ dimensiones [24].

La suma residual interna es la que permite la inicialización cercana a la identidad: si los parámetros de ambas proyecciones se inicializan con valores próximos a cero, el término $\mathrm{W}_{up}\, f(W_{down}\, h + b_{down}) + b_{up}$ es aproximadamente nulo y el módulo se reduce a $\mathrm{Adapter(h)} \approx h$ [Houlsby]. De este modo, la red original no se ve afectada al inicio del entrenamiento, y es durante este que los adapters se activan para modificar las activaciones a lo largo de la red.

2). Low-Rank Adaptation (LoRA):

A diferencia de los adapters, que introducen módulos nuevos en la red (B.1), LoRA pertenece a los métodos de reparametrización, es decir, no añade capas extra al modelo, en su lugar, restringe

la forma que puede tomar la actualización de los pesos ya existentes [10]. Este metodo se fundamenta en el comportamiento del rango $rank(W)$ de las matrices que se utilizaran para la actualización que se define como la dimensión del espacio generado por sus columnas, en otras palabras, el rango es el número máximo de columnas linealmente independientes que tiene una matriz.

La factorización de rango establece que toda matriz $\mathrm{M} \in \mathbb{R}^{d\times k}$ con $\mathrm{rank}(M) = r$ puede escribirse como el producto de dos matrices de dimensión intermedia r:$M = BA, \quad B \in \mathbb{R}^{d\times r}, \quad A \in \mathbb{R}^{r\times k}$ donde las r columnas de B forman una base del espacio columna de M, y cada columna de A contiene los coeficientes que expresan la columna correspondiente de M en esa base [25]. El número de valores necesarios para describir M pasa así de dk a $\mathrm{r(d + k)}$, una reducción considerable cuando $\mathrm{r} \ll \min(d, k)$.

Recíprocamente, cualquier producto de esta forma tiene rango a lo sumo r, ya que $\mathrm{rank}(BA) \leq \min(\mathrm{rank}(B), \mathrm{rank}(A)) \leq r$. De esta manera aunque la matriz sea formalmente de rango completo, su contenido relevante vive en un subespacio de dimensión r. Hu et al. [10] se inspiran en trabajos previos que muestran que los modelos preentrenados pueden adaptarse eficientemente aun cuando el entrenamiento se restringe a un subespacio de parámetros de dimensión reducida [10]. A partir de ello formulan la hipótesis central de LoRA: la actualización de los pesos durante la adaptación tiene un "rango intrínseco" bajo [10]. Es decir, para cada matriz preentrenada W_0, la actualización ΔW que la adapta a la tarea, aunque sea de la misma dimensión $\mathrm{d} \times k$, concentra su contenido relevante en unas pocas direcciones, de modo que se cumple:

$$\mathrm{rank}(\Delta W) = r \ll \min(d, k) \quad (8)$$

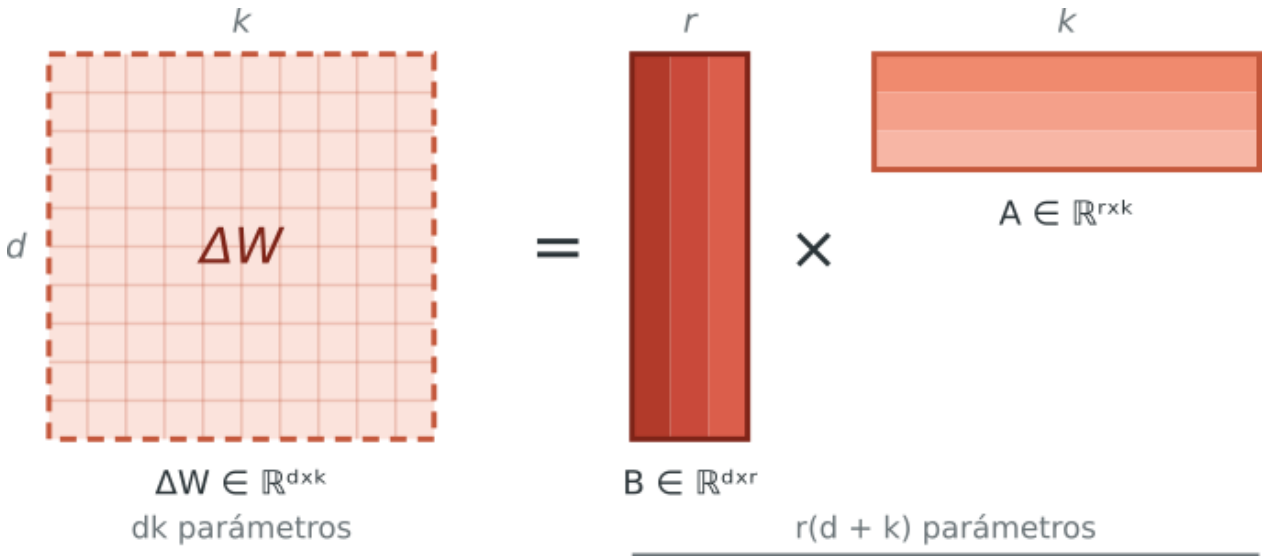


Fig. 5. Factorización de rango de la actualización de pesos ΔW en las matrices de bajo rango B y A.

Si esta hipótesis se cumple, no es necesario aprender ΔW entrada por entrada. Que esta matriz tenga rango r significa que todas sus columnas son combinaciones lineales de solo r vectores base. Por la factorización de rango, basta entonces con aprender dos matrices pequeñas: $\mathrm{B} \in \mathbb{R}^{d \times r}$, cuyas columnas son esos r vectores base, y $\mathrm{A} \in \mathbb{R}^{r \times k}$, que contiene, para cada columna de ΔW, los r coeficientes con los que se combinan dichos vectores. Su producto reconstruye la actualización completa:

$$\Delta W = BA \quad (9)$$

El número de parámetros a aprender pasa así de dk, (las entradas de ΔW), a $\mathrm{r(d + k)}$, las entradas de B y A. Por ejemplo, para $\mathrm{d = k = 1024}$ y $\mathrm{r = 8}$, esto supone pasar de 1.048.576 a 16.384 parámetros, aproximadamente el 1.6% del total.

Dada una matriz preentrenada $\mathrm{W_0} \in \mathbb{R}^{d \times k}$, LoRA la mantiene congelada y suma a su salida la contribución de la actualización de bajo rango $\Delta W = BA$, escalada por un factor α/r [10]:

$$h = W_0 x + \frac{\alpha}{r} BAx \quad (10)$$

donde B y A son los únicos parámetros entrenables. A se inicializa con valores gaussianos aleatorios y B con ceros, de modo que $\Delta W = 0$ al inicio y la capa reproduce exactamente su comportamiento preentrenado.

En principio, LoRA puede aplicarse a cualquier matriz densa de la red [10]. En una capa Transformer, las candidatas son las cuatro proyecciones del módulo de atención (W_q, W_k, W_v, W_o) y las dos del módulo feed-forward. Adaptar una proyección de atención, por ejemplo W_q, consiste en sustituir $\mathrm{W}_q x$ por $W_q x + \frac{\alpha}{r} B_q A_q x$, con $\mathrm{B}_q \in \mathbb{R}^{d_{model} \times r}$ y $ $A_q \in \mathbb{R}^{r \times d_{model}}$; el resto del mecanismo de atención no cambia, añadiendo al modelo $2\,\mathrm{r}\,\mathrm{d}_{model}$ parámetros.

IV. METODOLOGÍA

Este capítulo describe el procedimiento experimental seguido para evaluar el efecto de distintas estrategias de ajuste de un modelo de habla autosupervisado preentrenado sobre la clasificación binaria entre sujetos con enfermedad de Alzheimer (AD) y controles sanos (CTR) a partir de habla espontánea. La hipótesis central es que el ajuste del modelo a la tarea objetivo mejora el desempeño respecto a su uso como extractor congelado de características, en un escenario de datos etiquetados limitados.

La Figura 6 presenta el flujo general del sistema propuesto. Las grabaciones del corpus ADReSS 2020 se remuestrean a 16 kHz y se dividen en ventanas de duración fija. Cada ventana se procesa con wav2vec 2.0, sobre el cual se aplican las distintas estrategias de ajuste evaluadas. Las representaciones de la última capa se agregan mediante la media temporal, lo que produce un vector de dimensión 768 por ventana. Este vector se clasifica con un perceptrón multicapa (MLP) que reemplaza el *header* original del modelo. Finalmente, las predicciones de las ventanas de cada sujeto se combinan mediante votación mayoritaria para asignar la etiqueta final AD o CTR.

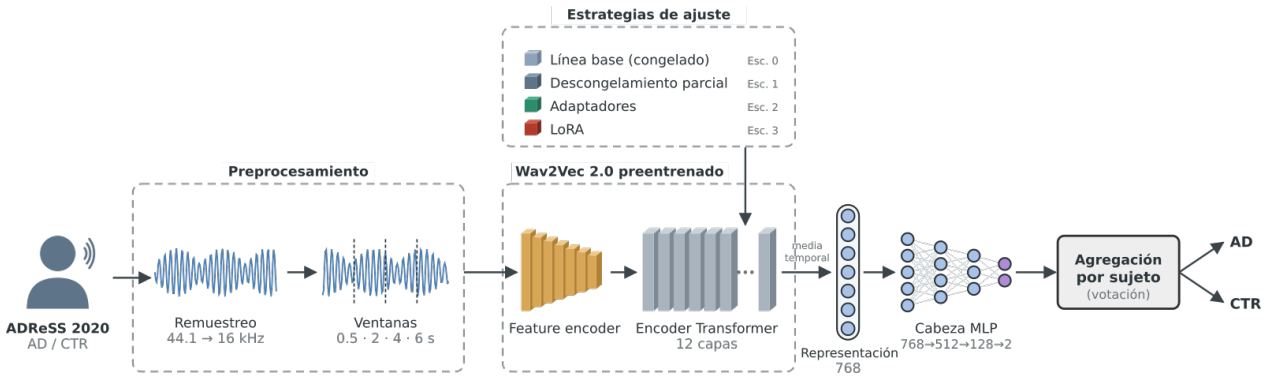


Fig. 6: Metodología implementada

Las estrategias de ajuste se organizaron de acuerdo a dos factores principales. El primero corresponde al mecanismo de ajuste, con tres niveles: descongelamiento parcial de las capas, inserción de módulos adaptadores de tipo *bottleneck* y adaptación de bajo rango (LoRA). El segundo corresponde al tipo de experimento, que define en qué capas del Transformer se aplica el ajuste, también con tres niveles:

- Estándar: el ajuste se distribuye en un conjunto fijo de capas equidistantes.
- Progresivo: el ajuste se aplica a las N últimas capas, con N = 1, …, 12.
- Inteligente: el ajuste se restringe a los bloques funcionales identificados por Pasad et al. (2021)[11].

Como condición de control se definió un escenario base, en el que el modeñp permanece completamente congelado y únicamente se entrena la etapa de clasificación. Todos los escenarios comparten los mismos datos, particiones, etapa de clasificación, protocolo de optimización de hiperparámetros y métricas de evaluación, de modo que las diferencias observadas son atribuibles únicamente a los factores manipulados.

A. Base de datos

1) Origen y tarea utilizada

Se utilizó la base de datos del reto *Alzheimer's Dementia Recognition through Spontaneous Speech* (ADReSS), presentado en Interspeech 2020 . Sus grabaciones provienen del Pitt Corpus de DementiaBank [26]. Los participantes describen oralmente la lámina *Cookie Theft* del Boston Diagnostic Aphasia Examination (BDAE), una tarea ampliamente usada en la evaluación del lenguaje en poblaciones con deterioro cognitivo.

A diferencia del Pitt Corpus completo, que presenta un desbalance entre grupos, ADReSS fue construido específicamente para estar balanceado por edad y género entre los sujetos con AD y los controles. Este balance reduce la posibilidad de que el clasificador aprenda variables

demográficas en lugar de características asociadas a la enfermedad, y convierte a ADReSS en un punto de referencia comparable entre estudios.

2) Composición y metadatos

La base de datos contiene 156 sujetos en total, divididos por los organizadores en una partición de entrenamiento y una de prueba, ambas balanceadas por clase (Tabla I).

TABLA I: DISTRIBUCIÓN DE SUJETOS EN LA BASE DE DATOS ADReSS

Partición	AD	CTR	Total
Entrenamiento	54	54	108
Prueba	24	24	48
Total	78	78	156

3) Preprocesamiento aplicado por los organizadores

Los organizadores del reto aplicaron tres etapas de preprocesamiento a las grabaciones originales:

1. Reducción de ruido estacionario sobre las grabaciones completas.
2. Segmentación por actividad de voz (VAD), con un algoritmo basado en un umbral de energía logarítmica de 65 dB y una duración máxima de 10 s por segmento.
3. Normalización del volumen de los segmentos resultantes.

Al contener únicamente segmentos con actividad de voz, esta versión elimina los silencios prolongados y las intervenciones sin habla, y garantiza un nivel de amplitud homogéneo entre sujetos, lo que reduce la influencia de las condiciones de grabación sobre la clasificación.

4) Partición para entrenamiento y validación

Para la optimización de hiperparámetros, la partición oficial de entrenamiento se dividió en un subconjunto de entrenamiento y uno de validación en proporción 80/20. Esta división se realizó mediante validación cruzada estratificada y agrupada (*StratifiedGroupKFold*) con cinco particiones y una semilla fija, tomando una de ellas como conjunto de validación. El resultado fue un conjunto de entrenamiento de 87 sujetos (45 AD y 42 CTR) y uno de validación de 21 sujetos (9 AD y 12 CTR). Dado que la estratificación se aplica a nivel de segmento, el balance por clase a nivel de sujeto es aproximado. Esta división se mantuvo idéntica en todos los escenarios experimentales, y la partición oficial de prueba se reservó exclusivamente para la evaluación final de los modelos seleccionados.

B. Preprocesamiento:

1) Remuestreo

Las grabaciones de ADReSS se encuentran muestreadas a 44.1 kHz, mientras que wav2vec 2.0 fue preentrenado con audio a 16 kHz. Dado que el *feature encoder* opera con una resolución temporal fija, en la que cada vector de salida representa aproximadamente 20 ms de señal, alimentar el modelo con una frecuencia de muestreo distinta a la de preentrenamiento altera la escala temporal de las representaciones. Por esta razón, cada segmento se convirtió a un único canal, promediando sus canales cuando la señal era estéreo, y se remuestreó de 44.1 kHz a 16 kHz aplicandose a cada segmento de forma individual, antes de su concatenación y enventanado.

2) Construcción de la señal por sujeto

Los segmentos entregados por los organizadores corresponden a fragmentos de habla activa de duración variable, con un máximo de 10 s. En lugar de enventanar cada segmento de forma

independiente, lo que descartaría la porción final de cada uno y eliminaría por completo los segmentos más cortos que la ventana, se construyó para cada sujeto una única señal continua mediante la concatenación de todos sus segmentos en orden cronológico.
El orden cronológico se recuperó a partir de las marcas temporales codificadas en el nombre de cada archivo. Esta construcción aprovecha la totalidad del habla disponible de cada sujeto, a costa de introducir uniones artificiales entre segmentos consecutivos, en las que se omiten las pausas eliminadas por la detección de actividad de voz.

3) Enventanado

La señal de cada sujeto se dividió en ventanas de duración fija con un traslape del $25\,\%$. La porción final de la señal que no alcanza a completar una ventana se descartó en lugar de completarse con ceros. Esta decisión se sustentó en un análisis de sensibilidad en el que se evaluó la inclusión de la ventana final incompleta cuando superaba un umbral mínimo de duración, rellenando el resto con ceros, dicha alternativa incrementaba el número de ventanas en aproximadamente un $2\,\%$, a cambio de introducir segmentos de silencio artificial que contaminan la representación. Como consecuencia adicional, todas las ventanas de un mismo experimento tienen exactamente la misma longitud, por lo que no se requiere relleno al conformar los *batchs* de entrenamiento.

4) Selección de la duración de las ventanas

Con el objetivo de evaluar el efecto del contexto temporal disponible para el modelo, se generaron conjuntos de ventanas con duraciones entre 0.5 s y 7.5 s. El límite inferior corresponde a una escala cercana a la duración de unidades fonéticas y silábicas, mientras que el límite superior está acotado por la duración total de habla del sujeto con menos habla, que es de 7.89 s. Cualquier ventana de 8 s o más implica la pérdida de sujetos completos, y dicha pérdida afecta de forma desproporcionada al grupo AD, lo que introduciría un sesgo en la composición de las particiones y afectaría la comparabilidad de las métricas a nivel de sujeto. Por esta razón, la duración máxima evaluada se fijó en 7.5 s, y se verificó para cada duración que la totalidad de los 156 sujetos aportara al menos una ventana. Dado el costo computacional de los escenarios de

ajuste, no todas las duraciones generadas se evaluaron en la totalidad de los experimentos. En su lugar, se realizó un barrido inicial con el escenario congelado sobre todas las duraciones de ventana, y se seleccionaron las duraciones de 6 s y 7.5 s según la exactitud del conjunto de prueba, [NO SE SI ESTO GENERE UN SESGO Y SEA MEJOR HACER EL ANALISIS POR VALIDACIÓN] las cuales se emplearon en todos los escenarios experimentales. El *baseline* se utilizó para esta selección por tratarse del escenario de menor costo y porque, al mantener el modelo congelado, refleja la calidad de las representaciones obtenidas con cada duración sin depender de la estrategia de ajuste. Adicionalmente, se evaluó como condición de referencia el uso directo de los *chunks* normalizados de ADReSS, sin concatenación ni enventanado, que corresponde al procedimiento habitual al trabajar con esta base de datos. Dado que los segmentos tienen longitudes distintas, al conformar los *batches* se completaron con ceros hasta la longitud del segmento más largo de cada uno y se empleó una máscara de atención para que las posiciones de relleno no intervinieran en el cálculo de las representaciones. Esta condición permite contrastar el efecto del enventanado propuesto frente al uso de los datos en su forma original.

C. Modelo preentrenado:

1) Modelos candidatos

Se evaluaron seis variantes preentrenadas de wav2vec 2.0 que difieren en su capacidad, en el volumen y la naturaleza de los datos de preentrenamiento, en la cobertura de idiomas y en si fueron posteriormente ajustadas para reconocimiento automático del habla (ASR). La Tabla II resume sus características principales.

El conjunto de candidatos permite contrastar tres ejes de variación: el tamaño del modelo (Base frente a Large), el volumen y la diversidad de los datos de preentrenamiento (desde 960 h de lectura en inglés hasta 436 000 h en 128 idiomas) y el efecto de un ajuste posterior para ASR sobre las representaciones.

TABLA II: MODELOS WAV2VEC 2.0 EVALUADOS

Modelo	Parámetros	Capas Transformer	Dimensión oculta	Datos de preentrenamiento	Ajuste para ASR	Idiomas
WAV2VEC2 BASE	~95 M	12	768	960 h (LibriSpeech)	No	Inglés
WAV2VEC2 ASR BASE 960H	~95 M	12	768	960 h (LibriSpeech)	960 h (LibriSpeech)	Inglés
WAV2VEC2 LARGE LV60K	~317 M	24	1024	60 000 h (Libri-Light)	No	Inglés
WAV2VEC2 ASR LARGE LV60K 960H	~317 M	24	1024	60 000 h (Libri-Light)	960 h (LibriSpeech)	Inglés
WAV2VEC2 XLSR 300M	~300 M	24	1024	436 000 h	No	128
VOXPOPULI ASR BASE 10K EN	~95 M	12	768	10 000 h (VoxPopuli)	543 h (VoxPopuli, inglés)	23 en preentrenamiento inglés en el ajuste

2) Criterio de selección

Dado que todas las estrategias de ajuste evaluadas en este trabajo se aplican sobre un único modelo preentrenado, la selección de dicho modelo constituye una etapa previa al diseño experimental. Para ello, los modelos candidatos se compararon completamente congelados y entrenando únicamente el *header* de clasificación, de modo que la dicha comparación refleje la

calidad de las representaciones de cada modelo para la tarea, sin que el resultado dependa de la estrategia de ajuste.

El modelo seleccionado fue wav2vec2-base-10k-voxpopuli-ft-en [27] . Este modelo emplea la arquitectura Base de wav2vec 2.0 que fue preentrenado de forma autosupervisada con 10 000 h de habla no etiquetada del corpus VoxPopuli, que reúne grabaciones de las sesiones del Parlamento Europeo en 23 idiomas, y posteriormente ajustado para ASR con 543 h de habla transcrita en inglés del mismo corpus.

NO SE SI MOSTRAR AQUI LOS RESULTADOS DE ESTOS MODELOS O DEJARLO PARA LA SECCIÓN SIGUIENTE.

D. Escenarios experimentales:

Todos los escenarios comparten la etapa de clasificación, la función de pérdida de entropía cruzada y el mismo espacio de búsqueda para los hiperparámetros: tres tasas de aprendizaje y tres coeficientes de regularización (weight decay). A este espacio se suman, en los mecanismos de adapters y LoRA, los hiperparámetros propios de cada uno. El entrenamiento se detuvo de forma anticipada cuando la pérdida en validación no mejoró en más de 1e-4 durante un número fijo de épocas consecutivas (paciencia), conservando los pesos de la época con menor pérdida.

1) Tipos de experimento:

Los tipos de experimento definen en qué capas del *encoder transformer* se aplica el ajuste, de forma independiente del mecanismo utilizado. Las capas se enumeran de 1 a 12, desde la más cercana a la entrada hasta la más cercana a la salida.

- Estándar: el ajuste se aplica simultáneamente en cuatro capas distribuidas uniformemente a lo largo del *encoder transformer* (capas 3, 6, 9 y 12). Este esquema corresponde a la práctica habitual de repartir la capacidad de adaptación a lo largo de toda la red, y genera una única configuración.

- Progresivo: el ajuste se aplica a las N últimas capas del *encoder transformer*, con N = 1, …, 12. N = 1 corresponde únicamente a la capa 12, y N = 12 a la totalidad de capas. Cada valor de N constituye una configuración independiente, entrenada a partir del modelo preentrenado original, para un total de 12 configuraciones.
- Inteligente: el ajuste se restringe a los bloques funcionales identificados por Pasad et al. (2021) en su análisis por capas de wav2vec 2.0: el bloque A, asociado a información fonética (capas 6 y 7); el bloque B, asociado a información semántica (capas 8 y 9); y el bloque C, asociado a la reconstrucción de la entrada (capas 10 a 12). Cada bloque constituye una configuración independiente, para un total de 3.

3) Descongelamiento parcial

En este mecanismo se descongelan los pesos originales de las capas seleccionadas, que se actualizan junto con la etapa de clasificación, mientras el resto del modelo permanece congelado. El *feature encoder* se mantuvo congelado en todos los casos, y la capa de proyección de características solo se descongeló en la configuración que descongela la totalidad del modelo (N = 12).

Dado que los pesos preentrenados son sensibles a actualizaciones de gran magnitud, se emplearon tasas de aprendizaje diferenciadas mediante dos grupos de parámetros. Para las capas descongeladas se fijó una tasa de 1e-5, fuera del espacio de búsqueda. El coeficiente de weight decay se compartió entre ambos grupos, de modo que cada configuración comprendió 9 combinaciones de hiperparámetros.

4) Adapters de tipo bottleneck

En este mecanismo, el modelo permanece completamente congelado y se insertan módulos adaptadores en las capas seleccionadas. Se evaluaron dos esquemas de inserción en los tres tipos de experimento, el de Pfeiffer et al. (2020) (Fig. 7) , con un adaptador por capa ubicado después de la red de alimentación directa (feed-forward), y el de Houlsby et al. (2019) (Fig. 8), con dos

adaptadores por capa, ubicados después del bloque de atención y después de la red feed-forward. Cada esquema se trató como una variante independiente, con su propia búsqueda de hiperparámetros.

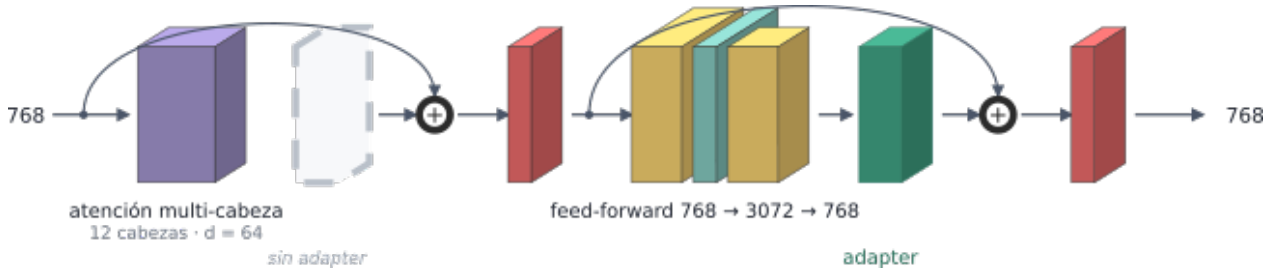


Fig. 7: Adaptador Bottleneck en configuración Pfeiffer

Cada adaptador proyecta la representación de dimensión d = 768 a una dimensión reducida $m = d/r_f$, donde r_f es el factor de reducción, tratado como hiperparámetro con valores $r_f \in \{8, 16, 32\}$, que corresponden a dimensiones de cuello de botella de 96, 48 y 24, respectivamente. Así, la búsqueda para cada configuración comprendió 27 combinaciones (3 tasas de aprendizaje × 3 coeficientes de weight decay × 3 factores de reducción). Cada adaptador utiliza una función de activación GELU y cuenta con 2dm + m + d parámetros, es decir, entre 37.656 y 148.320 según el factor de reducción.

Los adaptadores se inicializaron cerca de la identidad: la proyección de subida se inicializó en cero y la de bajada con una distribución normal de desviación estándar 1e-3, de modo que al inicio del entrenamiento el modelo reproduce exactamente las representaciones preentrenadas. Además de los adaptadores y el *header,* se entrenaron los parámetros de normalización de capa (layer normalization) posteriores a cada punto de inserción.

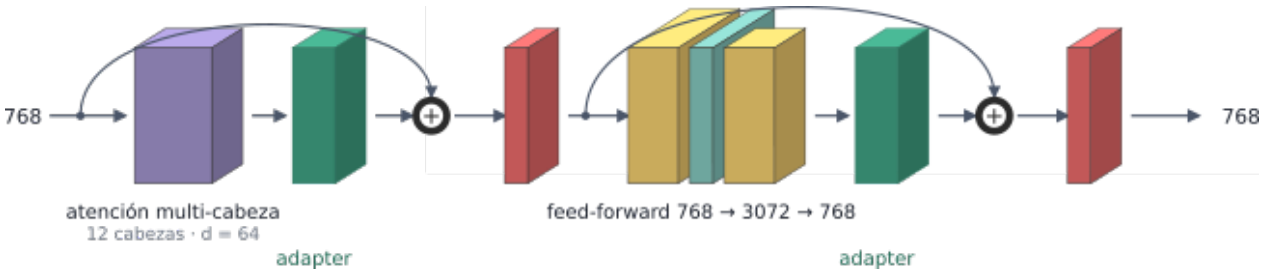


Fig. 8: Adaptador Bottleneck en configuración Houlsby

5) LoRA

En este mecanismo se añaden matrices de bajo rango a proyecciones lineales del mecanismo de atención de las capas seleccionadas. Las proyecciones intervenidas se trataron como una variable del diseño, con siete variantes. En primer lugar, se evaluaron las proyecciones de consulta (q), clave (k), valor (v) y salida (o) de forma individual, con el fin de aislar la contribución de cada una. En segundo lugar, se evaluaron tres combinaciones de dos proyecciones. La combinación qv corresponde a la configuración propuesta en el trabajo original de LoRA, en el que la adaptación conjunta de las proyecciones de consulta y valor obtuvo el mejor desempeño bajo un presupuesto fijo de parámetros [21]. Las combinaciones qk y vo se definieron a partir de la descomposición funcional del mecanismo de atención propuesta por Elhage et al. (2021) [28] en el que se expone que las proyecciones de consulta y clave conforman el circuito que determina de qué posiciones de la secuencia se toma la información, mientras que las proyecciones de valor y salida conforman el circuito que determina qué información se transporta desde dichas posiciones y cómo se incorpora a la representación de salida de la capa. De este modo, la combinación qk permite adaptar únicamente la forma en que se distribuye la atención, y la combinación vo únicamente el contenido que la atención transmite. Cada variante se trató como una configuración independiente, con su propia búsqueda de hiperparámetros.

Para cada variante se exploró el rango $r \in 4, 8, 16$, con un factor de escala $\alpha = 2r$, de modo que la razón $\alpha / r = 2$ se mantuvo constante. Esto da 27 combinaciones de hiperparámetros por configuración. Se aplicó un dropout de 0.1 sobre la entrada de la rama de bajo rango. La matriz B se inicializó en cero y la matriz A con la inicialización uniforme de Kaiming, siguiendo la

implementación de referencia de LoRA, de modo que la salida del modelo al inicio del entrenamiento coincide con la del modelo preentrenado.

E. Etapa de clasificación

La representación de cada ventana se clasifica mediante un perceptrón multicapa con dos capas ocultas de 512 y 128 neuronas y función de activación ReLU, seguido de una capa de salida de dos neuronas.

Esta etapa cuenta con 459.650 parámetros. La probabilidad de que la ventana pertenezca a la clase AD se obtiene aplicando la función softmax sobre la salida y se entrenó minimizando la entropía cruzada entre la distribución predicha y la etiqueta de la ventana, sin ponderación de clases. La misma arquitectura se utilizó en todos los escenarios experimentales, y se inicializó aleatoriamente en cada entrenamiento. En el escenario *baseline*, el *header* se entrena sobre las representaciones extraídas previamente del codificador congelado; en los demás escenarios, se entrena conjuntamente con los parámetros del modelo que se ajustan en cada caso.

F. Selección de modelo y evaluación

Para cada configuración experimental, y de forma independiente para cada duración de ventana, se realizó una búsqueda exhaustiva en rejilla sobre el espacio de hiperparámetros del mecanismo correspondiente (Tabla III). Se seleccionó la combinación con mayor exactitud a nivel de sujeto en el subconjunto de validación, calculada a partir de la probabilidad promedio de la clase AD sobre las ventanas de cada sujeto; este criterio se prefirió a la votación mayoritaria porque, con 21 sujetos de validación, esta última presenta frecuentes empates entre combinaciones. Con los hiperparámetros seleccionados se entrenó un nuevo modelo con semilla fija, que constituye el modelo final de la configuración. La partición de prueba no intervino en la búsqueda ni en la selección, y cada modelo final se evaluó sobre ella una única vez.

En la evaluación, la etiqueta de cada sujeto se asignó mediante votación mayoritaria sobre las predicciones de sus ventanas. El desempeño se reporta a nivel de sujeto sobre los 48 sujetos de prueba, considerando AD como clase positiva, mediante la exactitud, la sensibilidad, la especificidad, el F1 y el AUC, este último calculado a partir de la probabilidad promedio de la clase AD por sujeto.

V. ANÁLISIS DE RESULTADOS

A. Referencia: Segmentos originales de ADReSS

El análisis de resultados parte de la condición más directa posible: evaluar el baseline sobre los segmentos de audio tal como los distribuye el corpus orginal, sin ningún procesamiento adicional. Este resultado cumple dos funciones. Por un lado, permite ubicar el desempeño del modelo frente al *baseline* acústico reportada por los organizadores del reto. Por otro lado, establece el punto de comparación contra el cual se mide el efecto del enventanado y de las estrategias de ajuste evaluadas en las secciones siguientes. La Tabla III resume las métricas obtenidas sobre la partición de prueba.

TABLA III: DESEMPEÑO SOBRE LOS SEGMENTOS ORIGINALES DE ADReSS

Escenario	Exactitud	Sensibilidad	Especificidad	F1	AUC
Baseline	64.58%	83.33%	45.83%	70.18%	0.802

La exactitud obtenida (64.58 %) es comparable con el desempeño obtenido en el reto original de 62.5% [8]. Sobre los segmentos originales, las representaciones congeladas de wav2vec 2.0 no ofrecen una ventaja sustancial frente a los descriptores acústicos clásicos.

El aspecto más relevante es el desbalance entre sensibilidad y especificidad. El modelo identifica correctamente 20 de los 24 sujetos con AD, pero solo 11 de los 24 controles, de modo que más de la mitad de los controles son clasificados como pacientes.

B. Efecto de la duración de la ventana

Una vez establecido el desempeño sobre los segmentos originales, se evaluó cómo cambia el comportamiento del modelo completamente congelado cuando estos se sustituyen por ventanas sintéticas de duración constante. Este barrido permite identificar las duraciones de ventana en las

que las representaciones de wav2vec 2.0 contienen información suficiente para discriminar entre pacientes con AD y controles, y con ello definir las condiciones sobre las que se evalúan las estrategias de ajuste. La Tabla IV presenta las metricas obtenidas para cada duración.

TABLA IV: DESEMPEÑO DEL BASELINE POR TAMAÑO DE VENTANA

Tamaño de ventana	Exactitud	Sensibilidad	Especificidad	F1	AUC
0.5 s	50.00%	0.00%	100.00%	0.00%	0.2864583333
2 s	50.00%	0.00%	100.00%	0.00%	0.4565972222
4 s	50.00%	0.00%	100.00%	0.00%	0.5503472222
6 s	66.67%	58.33%	75.00%	63.64%	0.7048611111
7.5s	72.92%	66.67%	79.17%	71.11%	0.7552083333

Los resultados muestran dos regímenes claramente diferenciados. Para ventanas de 0.5 , 2 y 4 s, la exactitud es exactamente 50.0%, que corresponde al nivel de azar en una partición de prueba balanceada. En estas condiciones el clasificador colapsa hacia una sola clase y asigna la misma etiqueta a todos los sujetos. Este comportamiento indica que ventanas de hasta 4 s no contienen información suficiente para que el modelo aprenda una frontera de decisión útil. A partir de ventanas de 6 s la exactitud alcanza 66.7%, ligeramente por encima de la obtenida con los segmentos originales (64.6%). Con ventanas de 7.5 s llega a 72.9%, lo que representa cuatro sujetos adicionales correctamente clasificados respecto a la condición de referencia.
La selección de las ventanas de 6 y 7.5 s se sustenta en tres criterios.

El primero es que el modelo inicial debe ser una referencia válida. En las ventanas de 0.5 a 4s, el *baseline* colapsa, y cualquier mejora obtenida por una estrategia de ajuste en esas condiciones reflejaría principalmente la salida del colapso y no la contribución de la adaptación del modelo. Solo en 6 y 7.5 s se produce una frontera de decisión informativa contra la cual la comparación tiene sentido.

El segundo es que la condición de la ventana debe mejorar la referencia de los segmentos originales. Ambas ventanas superan la exactitud obtenida con los segmentos originales . Además, como se puede observar en La Tabla IV, reducen el sesgo hacia la clase AD observado en la sección anterior: la especificidad pasa de 45.8% a 75.0% en 6s y a 79.2% en 7.5s.

El tercero es que las conclusiones no deben depender de una única duración. La ventana de 7.5s corresponde al mejor desempeño del *baseline*, mientras que la de 6s es la duración mínima a partir de la cual el modelo deja de colapsar. Evaluar ambas permite verificar si el comportamiento de las estrategias de ajuste se mantiene a lo largo de diferentes analisis, en lugar de reportar hallazgos que podrían ser específicos de una sola condición.

C. Resultados con ventana de 6 segundos

1) Panorama general:

En esta sección se comparan todas las estrategias de ajuste evaluadas con ventanas de 6 s frente a su propio *baseline.* La Fig. 9 presenta la exactitud obtenida por cada configuración sobre la partición de prueba, agrupada por familia de método, mientras que La Tabla V presenta los resultados generales obtenidos para todos los experimentos realizados con esta duración de ventana.

Podemos observar que el modelo original congelado alcanza una exactitud de 66.7 %. Todas las configuraciones progresivas e inteligentes la superan, con valores entre 68.8 % y 83.3 %. En cambio, las dos configuraciones estándar quedan por debajo: 64.6 % para los adaptadores *bottleneck* y 62.5 % para LoRA.

El contraste con las configuraciones estándar es el resultado más informativo de esta comparación. El adaptador bottleneck estándar y el progresivo tienen presupuestos de parámetros casi idénticos (304,320 y 313,536), pero sus exactitudes difieren en 12.5 puntos porcentuales, es decir, seis sujetos. En LoRA el caso es aún más claro: las configuraciones estándar y progresiva entrenan exactamente el mismo número de parámetros (24,576) y difieren en 14.6 puntos porcentuales, siete sujetos. Por lo tanto, la capacidad de adaptación, entendida como número de parámetros entrenables, no explica la diferencia de desempeño.

TABLA V: RESULTADO DE EXPERIMENTOS PARA VENTANAS DE 6 SEGUNDOS

Escenario	Configuración	Capas	Exactitud (%)	Sensibilidad (%)	Especificidad (%)	F1 (%)	AUC
Baseline	Modelo congelado	—	66.7	58.3	75	63.6	0.705
Descongelamiento progresivo	N = 4	9–12	72.9	66.7	79.2	71.1	0.748
Descongelamiento inteligente	Bloque B (semántico)	8–9	68.8	79.2	58.3	71.7	0.842
Bottleneck estándar	Pfeiffer	3, 6, 9, 12	64.6	79.2	50	69.1	0.668
Bottleneck progresivo	Houlsby, N = 4	9–12	77.1	79.2	75	77.6	0.868
Bottleneck inteligente	Houlsby, bloque B (semántico)	8–9	83.3	83.3	83.3	83.3	0.873
LoRA estándar	Matriz k	3, 6, 9, 12	62.5	62.5	62.5	62.5	0.686
LoRA progresivo	Matriz v, N = 1	12	77.1	70.8	83.3	75.6	0.811
LoRA inteligente	Matrices v y o, bloque C (reconstructivo)	10–12	79.2	75	83.3	78.3	0.835

Lo que distingue a las configuraciones exitosas es la ubicación de la adaptación: las estándar distribuyen los módulos en capas fijas a lo largo de toda la red, mientras que las progresivas e inteligentes los concentran en las capas intermedias y superiores, seleccionadas según su contenido representacional. En el caso de LoRA, la diferencia combina la ubicación y la matriz adaptada.

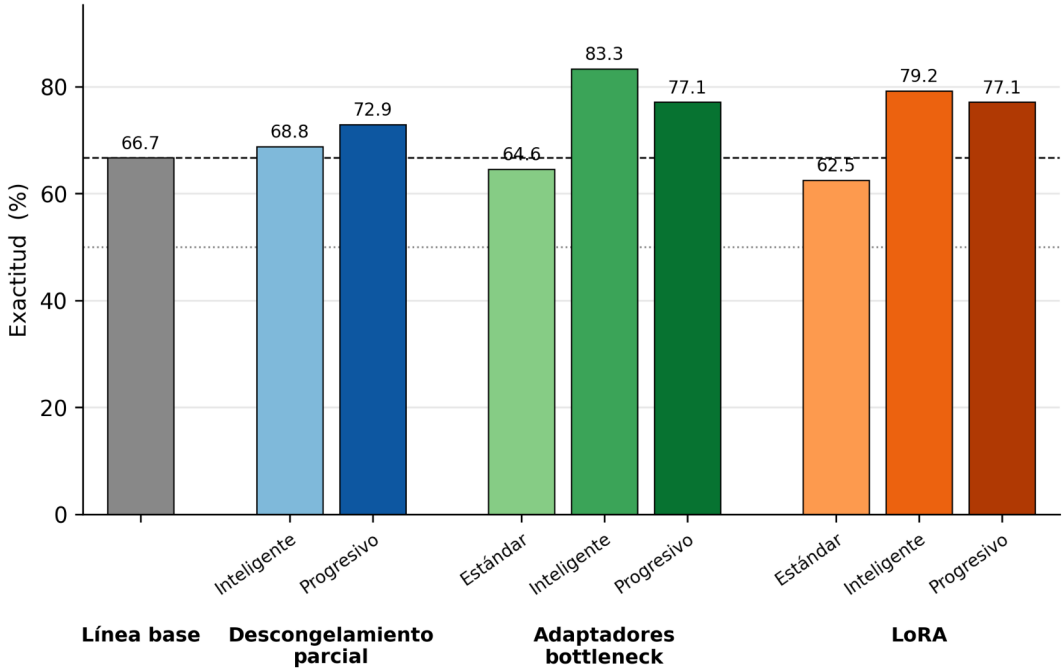


Fig. 9:Exactitud de modelos para ventanas de 6 segundos

El descongelamiento parcial muestra un comportamiento distinto al de los métodos basados en módulos adicionales. Sus dos variantes superan el baseline con mejoras modestas (68.8 % y 72.9 %), pese a entrenar entre 14 y 28 millones de parámetros. Esto es coherente con el sobreajuste temprano observado durante su entrenamiento. Con 108 sujetos de entrenamiento, modificar directamente millones de pesos del modelo preentrenado ofrece una complejidad muy grande para un espacio de representación tan pequeño. Además, en esta familia la variante progresiva supera a la inteligente, a diferencia de lo que ocurre con los adaptadores y LoRA.

2) mejores configuraciones:

Para analizar con mayor detalle el comportamiento de los modelos, se seleccionó la configuración de mayor exactitud de cada familia. La VI presenta sus métricas, y la Fig. 10 muestra sus curvas ROC (a) junto con la distribución de puntajes del mejor modelo (b).

TABLA VI: MEJORES RESULTADOS PARA 6 SEGUNDOS

Escenario	Configuración	Capas	Exactitud (%)	Sensibilidad (%)	Especificidad (%)	F1 (%)	AUC
Línea base	Modelo congelado	—	66.7	58.3	75	63.6	0.705
Descongelamiento progresivo	N = 4	9–12	72.9	66.7	79.2	71.1	0.748
Bottleneck inteligente	Houlsby, bloque B (semántico)	8–9	83.3	83.3	83.3	83.3	0.873
LoRA inteligente	Matrices v y o, bloque C (reconstructivo)	10–12	79.2	75	83.3	78.3	0.835

El adaptador bottleneck inteligente, con módulos Houlsby en el bloque semántico (capas 8 y 9), obtiene el mejor desempeño: 83.3 % en exactitud, sensibilidad, especificidad y F1, con un AUC de 0.873. La mejora es simultánea en ambas clases: la sensibilidad pasa de 58.3 % a 83.3 % y la especificidad de 75.0 % a 83.3 % respecto al modelo congelado manteniendose perfectamente balanceado.

La distribución de puntajes del adaptador bottleneck inteligente (Fig. 10b) muestra que el modelo separa a la mayoría de los sujetos con un margen amplio respecto al umbral. De los 24 controles, 14 reciben puntajes inferiores a 0.31, y de los 24 pacientes con AD, 17 se ubican por encima de 0.54, con una concentración marcada entre 0.6 y 0.77. Para estos sujetos, la decisión del modelo no depende del valor exacto del umbral.

Las errores se reparten de forma equilibrada entre ambas clases, el número de pacientes con puntajes bajos es similar al de controles con puntajes altos. Esto explica que la sensibilidad y la especificidad sean idénticas (83.3 %) y confirma que el modelo no favorece a ninguna de las dos clases.

Este resultado puede interpretarse a partir del funcionamiento del adaptador. Cada módulo bottleneck recibe la representación producida por una subcapa, la proyecta a un espacio de menor dimensión, la transforma y la devuelve a la dimensión original, sumándola a la representación de entrada mediante una conexión residual. En la práctica, el adaptador actúa como una traducción entre la representación de una capa y la que recibe la siguiente, conserva la información original y agrega solo la corrección necesaria para la tarea. Ubicada en el bloque semántico, permite reorientar hacia la clasificación de AD las representaciones del modelo, sin modificar los pesos que las generan.

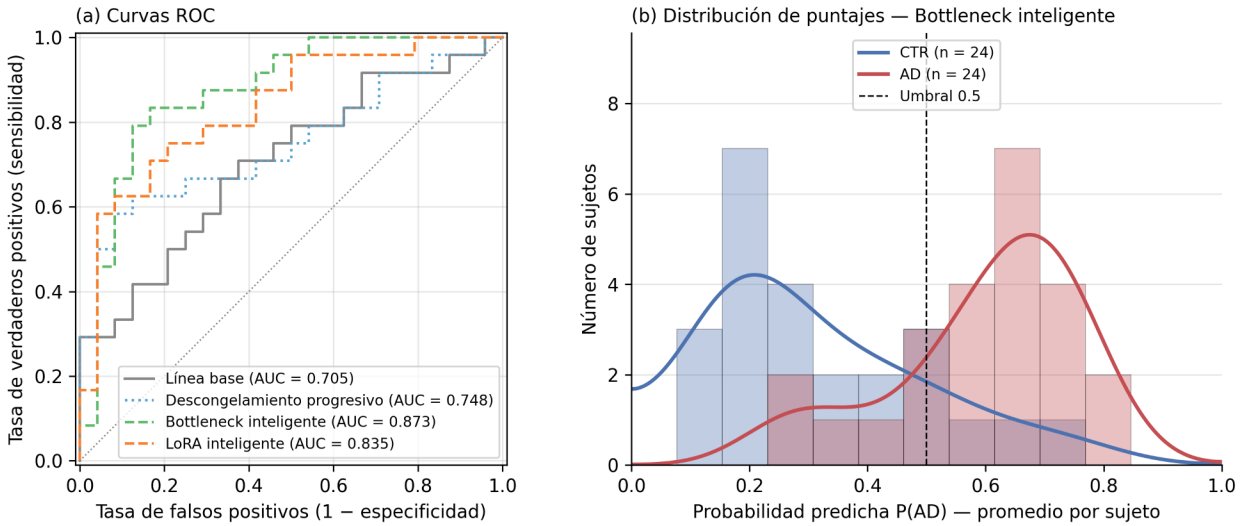


Fig. 10: Curvas ROC y diagramas de distribución para el mejor modelo de 6 seg

Las curvas ROC (Fig. 10a) muestran que la ventaja del adaptador bottleneck inteligente no depende del umbral de decisión. Su curva se ubica por encima del *baseline* en prácticamente todo el rango. Con una tasa de falsos positivos cercana al 17 %, detecta alrededor del 83 % de los pacientes con AD, mientras que el modelo completamente congelado con la misma tasa, detecta aproximadamente el 42 %. LoRA inteligente sigue un comportamiento similar (AUC de 0.835). Ambos modelos mejoran la separación entre clases en toda la población, y no solo en torno al punto de operación elegido.

El descongelamiento progresivo muestra un patrón distinto. Su curva supera a la del *baseline* solo cuando la tasa de falsos positivos es baja, por debajo del 10 % aproximadamente, donde detecta cerca del 60 % de los pacientes. A partir de una tasa cercana al 35 %, ambas curvas se superponen. Esto indica que el descongelamiento separa mejor a los sujetos con puntajes más extremos, los casos más evidentes de cada clase, pero no mejora la discriminación de los sujetos con puntajes intermedios, que son los que determinan el desempeño en el resto de la curva. Su AUC (0.748), apenas superior (0.705), refleja esta mejora parcial.

3) Eficiencia de parametros:

El ultimo punto del análisis relaciona el desempeño de cada configuración con el número de parámetros del modelo base que modifica. La Tabla VII presenta el número de parámetros entrenables de cada configuración, su proporción respecto al total del modelo y la reducción frente al ajuste fino completo.

TABLA VII: PARAMETROS ENTRENABLES POR CONFIGURACIÓN EN VENTANAS DE 6 SEGUNDOS

Método	Configuración	Capas	Parámetros entrenables	% del modelo	Reducción (×)	Exactitud (%)
Línea base	Modelo congelado + MLP	—	0	0	—	66.7
Descongelamiento progresivo	N = 4	9–12	28,351,488	30.04	3.3	72.9
Descongelamiento inteligente	Bloque B (semántico)	8–9	14,175,744	15.02	6.7	68.8
Bottleneck estándar	Pfeiffer	3, 6, 9, 12	304,320	0.322	310	64.6
Bottleneck progresivo	Houlsby, N = 4	9–12	313,536	0.332	301	77.1
Bottleneck inteligente	Houlsby, bloque B (semántico)	8–9	156,768	0.166	602	83.3
LoRA estándar	Matriz k	3, 6, 9, 12	24,576	0.026	3,840	62.5
LoRA progresivo	Matriz v, N = 1	12	24,576	0.026	3,840	77.1
LoRA inteligente	Matrices v y o, bloque C (reconstructivo)	10–12	147,456	0.156	640	79.2

Si la cantidad de parámetros fuera determinante, los puntos deberían ascender de izquierda a derecha. Ocurre lo contrario: las dos variantes de descongelamiento parcial, que entrenan entre 14 y 28 millones de parámetros, se ubican entre las de menor exactitud, mientras que el mejor resultado (83.3 %) lo obtiene el adaptador bottleneck inteligente con apenas 156,768 parámetros.

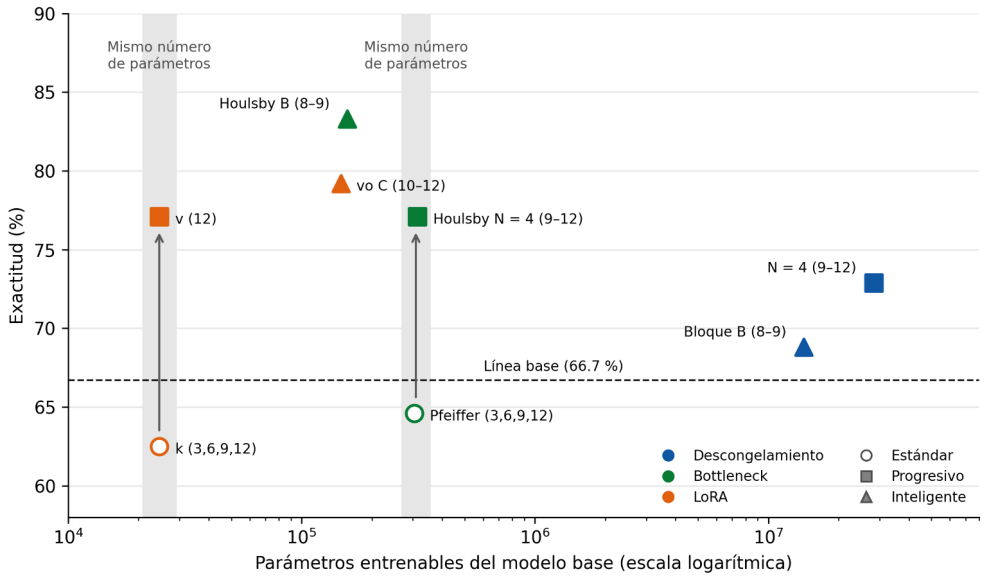


Fig. 11: Exactitud vs Numero de parámetros de cada modelo en ventanas de 6 segundos

La evidencia más directa se encuentra en las franjas sombreadas, que agrupan configuraciones con el mismo presupuesto de parámetros. En LoRA, las configuraciones estándar y progresiva entrenan exactamente 24,576 parámetros, y la exactitud pasa de 62.5 % a 77.1 %. En los adaptadores bottleneck, con cerca de 300,000 parámetros en ambos casos, pasa de 64.6 % a 77.1 %. Con igual número de parámetros, lo único que cambia es la ubicación de la adaptación. Por lo tanto, en condiciones de datos limitados, reducir el número de parámetros entrenables no basta, la eficiencia se obtiene cuando la adaptación se ubica en las capas cuyo contenido es relevante para la tarea.

C. Resultados con ventana de 7.5 segundos

1) Panorama general:

En esta sección se comparan las estrategias de ajuste evaluadas con ventanas de 7.5 s frente a su propio baseline. La Fig. 12 presenta la exactitud obtenida por cada configuración, agrupada por familia de método, y la Tabla VIII presenta los resultados de todos los experimentos realizados con esta duración de ventana.

Con esta duración, el modelo congelado alcanza una exactitud de 72.9 %, un punto de partida más exigente que el de la ventana de 6 s. Aun así, todas las configuraciones progresivas e inteligentes de los adaptadores bottleneck y de LoRA lo superan, con valores entre 75.0 % y 77.1%.

Las configuraciones estándar vuelven a quedar rezagadas. El adaptador bottleneck estándar obtiene la menor exactitud de todos los experimentos (60.4 %). LoRA estándar iguala al baseline en exactitud (72.9 %), pero esta cifra oculta un peor comportamiento: su sensibilidad cae a 54.2%, es decir, deja de detectar a casi la mitad de los pacientes, y su AUC (0.668) es inferior al del baseline (0.755). Su exactitud se sostiene únicamente porque clasifica a la mayoría de los sujetos como controles. En contraste, LoRA inteligente, que entrena exactamente el mismo número de parámetros, alcanza 77.1 % con una sensibilidad de 79.2 %. Como en la ventana de 6s, la diferencia entre ambas configuraciones no está en cuántos parámetros se entrenan, sino en dónde se ubica la adaptación.

Tabla VIII: TABLA V: RESULTADO DE EXPERIMENTOS PARA VENTANAS DE 7.5 SEGUNDOS

Escenario	Configuración	Capas	Exactitud (%)	Sensibilidad (%)	Especificidad (%)	F1 (%)	AUC
Baseline	Modelo congelado	—	72.9	66.7	79.2	71.1	0.755
Descongelamiento progresivo	N = 6	7–12	68.8	66.7	70.8	68.1	0.684
Descongelamiento inteligente	BloqueC (reconstructivo)	10–12	70.8	54.2	87.5	65.0	0.677
Bottleneck estándar	Pfeiffer	3,6,9,12	60.4	62.5	58.3	61.2	0.627
Bottleneck progresivo	Pfeiffer, N = 9	4–12	77.1	62.5	91.7	73.2	0.802
Bottleneck inteligente	Houlsby,bloqueA (fonético)	6–7	75.0	75.0	75.0	75.0	0.776
LoRA estándar	Matriz k	3,6,9,12	72.9	54.2	91.7	66.7	0.668
LoRA progresivo	Matrices q y k, N = 5	8–12	77.1	66.7	87.5	74.4	0.747
LoRA inteligente	Matrices v y o, bloque B (semántico)	8–9	77.1	79.2	75.0	77.6	0.818

El descongelamiento parcial presenta el cambio más notable respecto a la ventana de 6 s, sus dos variantes quedan por debajo del baseline (68.8 % y 70.8 %), a pesar de entrenar entre 21 y 42 millones de parámetros. Una posible explicación es que las ventanas más largas producen menos ejemplos de entrenamiento por sujeto. Esto agrava el riesgo de sobreajuste en los métodos que modifican directamente millones de pesos del modelo, mientras que los adaptadores y LoRA, con presupuestos varios órdenes de magnitud menores, mantienen su ventaja.

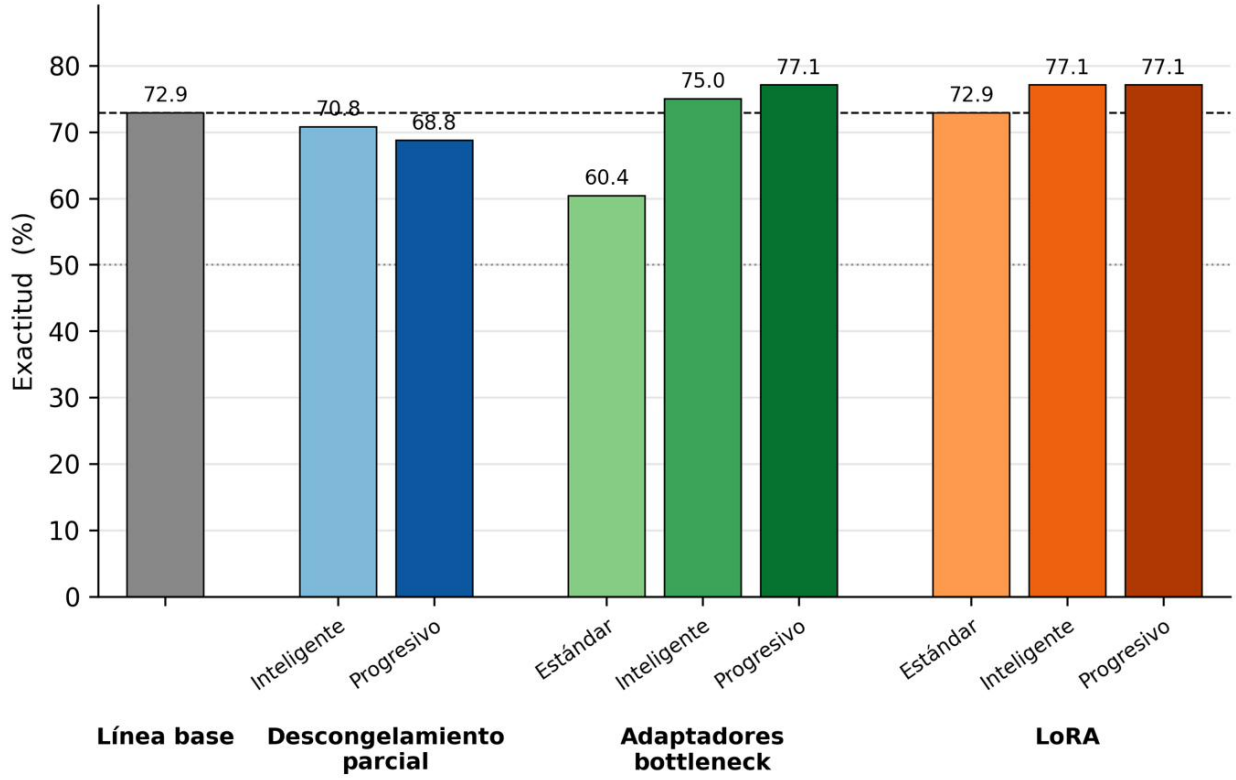


Fig. 12. Exactitud de modelos para ventanas de 7.5 segundos

2) Mejores configuraciones:

La Tabla IX presenta la configuración de mayor exactitud de cada familia, y la Fig. 13 muestra sus curvas ROC (a) junto con la distribución de puntajes del mejor modelo (b). Tres configuraciones comparten la mayor exactitud (77.1 %): el adaptador bottleneck progresivo, LoRA progresivo y LoRA inteligente. Entre ellas se destaca LoRA inteligente, con matrices v y o en el bloque semántico (capas 8 y 9), por obtener el mayor AUC (0.818) y el comportamiento más balanceado entre clases.

Tabla IX: TABLA VI: MEJORES RESULTADOS PARA 7.5 SEGUNDOS

Escenario	Configuración	Capas	Exactitud (%)	Sensibilidad (%)	Especificidad (%)	F1 (%)	AUC
Baseline	Modelo congelado	—	72.9	66.7	79.2	71.1	0.755
Descongelamiento inteligente	Bloque C (reconstructivo)	10–12	70.8	54.2	87.5	65.0	0.677
Bottleneck progresivo	Pfeiffer, N = 9	4–12	77.1	62.5	91.7	73.2	0.802
LoRA inteligente	Matrices v y o, bloque B (semántico)	8–9	77.1	79.2	75.0	77.6	0.818

La forma en que cada modelo supera al baseline es distinta. El adaptador bottleneck progresivo alcanza una especificidad de 91.7 %, pero su sensibilidad (62.5 %) es inferior a la del baseline: su ganancia proviene de reconocer mejor a los controles, no de detectar más pacientes. LoRA inteligente hace lo contrario: eleva la sensibilidad de 66.7 % a 79.2 % a cambio de una leve reducción en la especificidad (de 79.2 % a 75.0 %), lo que lo convierte en el modelo más útil para identificar sujetos con AD.

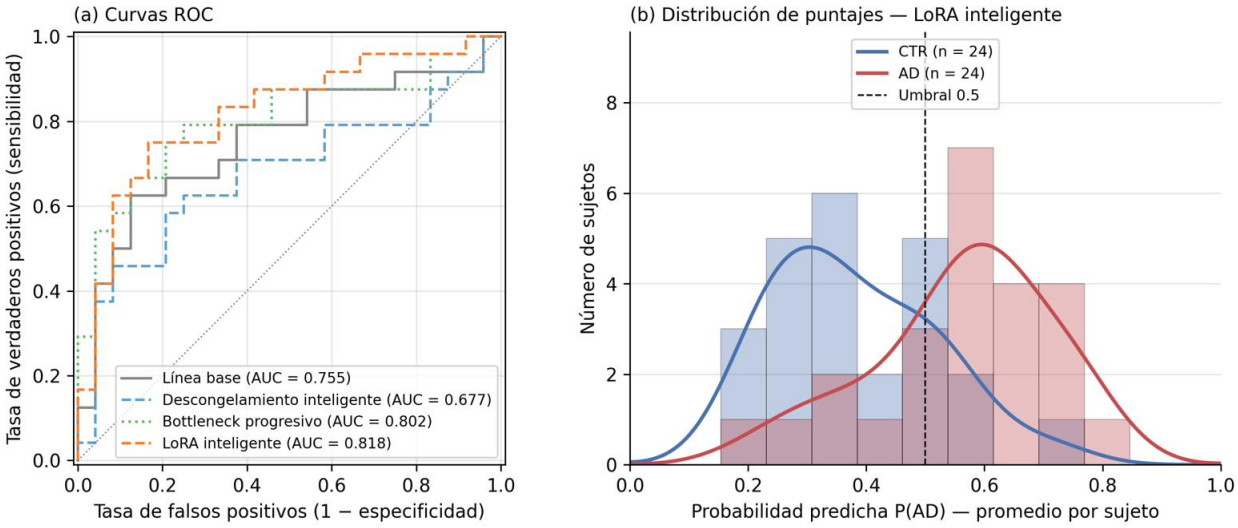


Fig. 13: Curvas ROC y diagramas de distribución para el mejor modelo de 7.5 segundos

Las curvas ROC (Fig. 13a) muestran que, con esta ventana, las diferencias entre modelos son más estrechas que con 6 s. LoRA inteligente y el adaptador bottleneck progresivo se separan del baseline principalmente en la región intermedia de tasas de falsos positivos, con una tasa cercana al 17 %, LoRA inteligente detecta alrededor del 75 % de los pacientes, frente a aproximadamente el 62 % del modelo congelado. El descongelamiento inteligente, en cambio, queda por debajo del baseline en la mayor parte del rango (AUC de 0.677). A diferencia de lo observado en 6 s, su menor exactitud no se debe al umbral de decisión, sino a una capacidad de discriminación realmente inferior a la del modelo congelado.

La distribución de puntajes de LoRA inteligente (Fig. 13b) muestra a los controles concentrados alrededor de 0.3 y a los pacientes alrededor de 0.6. Los dos picos están más cercanos que en el mejor modelo de 6 s, y la región de superposición es más amplia, entre 0.15 y 0.55 aproximadamente. Esto explica la mayor cantidad de errores de clasificación y el menor AUC respecto al adaptador bottleneck inteligente de la ventana de 6 s.

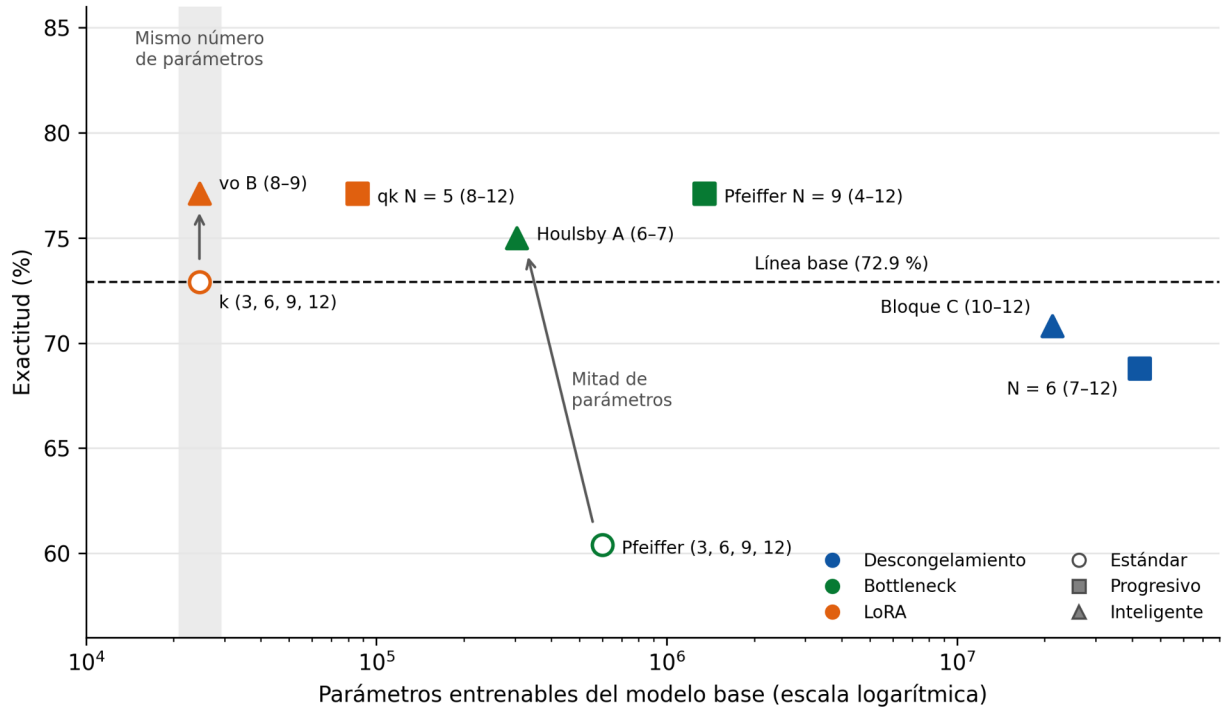
3) Eficiencia de parámetros:

La Tabla X presenta el número de parámetros entrenables de cada configuración con ventana de 7.5 s, y la Fig. 14 los relaciona con la exactitud obtenida.

Con esta duración de ventana, la relación entre parámetros y desempeño es incluso más clara que con 6 s. Las dos configuraciones con más parámetros, las de descongelamiento parcial, son las únicas familias que no logran superar al baseline. En el extremo opuesto, LoRA inteligente alcanza la mayor exactitud (77.1 %) entrenando apenas 24,576 parámetros, el 0.026 % del modelo, el mismo resultado que el adaptador bottleneck progresivo con 55 veces más parámetros.

Método	Configuración	Capas	Parámetros entrenables	% del modelo	Reducción (×)	Exactitud (%)
Baseline	Modelo congelado	—	0	0	—	72.9
Descongelamiento progresivo	N = 6	7–12	42,527,232	45.06	2.2	68.8
Descongelamiento inteligente	Bloque C (reconstructivo)	10–12	21,263,616	22.53	4.4	70.8
Bottleneck estándar	Pfeiffer	3, 6, 9, 12	599,424	0.635	157	60.4
Bottleneck progresivo	Pfeiffer, N = 9	4–12	1,348,704	1.429	70	77.1
Bottleneck inteligente	Houlsby, bloque A (fonético)	6–7	304,320	0.322	310	75.0
LoRA estándar	Matriz k	3, 6, 9, 12	24,576	0.026	3,840	72.9
LoRA progresivo	Matrices q y k, N = 5	8–12	86,016	0.091	1,097	77.1
LoRA inteligente	Matrices v y o, bloque B (semántico)	8–9	24,576	0.026	3,840	77.1

Las configuraciones estándar confirman el patrón. En LoRA, las configuraciones estándar e inteligente entrenan exactamente el mismo número de parámetros, y la exactitud pasa de 72.9 % a 77.1 %, con un AUC que sube de 0.668 a 0.818. En los adaptadores bottleneck el contraste es mayor: la configuración estándar, con 599,424 parámetros, obtiene 60.4 %, mientras que la inteligente, con la mitad de parámetros, alcanza 75.0 %. Es decir, ubicar bien la adaptación con menos parámetros supera a ubicarla mal con más.



E. Consistencia de las matrices v y o en LoRA

Las secciones anteriores mostraron que la configuración ganadora de los adaptadores bottleneck cambia entre ventanas: el bloque B (semántico) en 6 s y el bloque A (fonético) en 7.5 s. En LoRA, en cambio, hay un elemento que se mantiene: las matrices de la atención que se adaptan. Esta sección analiza ese patrón a partir de las configuraciones LoRA progresivas e inteligentes de ambas ventanas, resumidas en la Tabla XI.

Las tres configuraciones que incluyen la matriz v superan el AUC del baseline en su respectiva ventana. La única configuración que no adapta la matriz v, LoRA progresivo con ventana de 7.5 s, que adapta q y k, queda por debajo del AUC del baseline.

Tabla X: CONFIGURACIONES LoRA POR MATRICES ADAPTADAS

Ventana	Escenario	Matrices	Capas	r	Parámetros entrenables	Exactitud (%)	AUC	ΔAUC vs. baseline
6 s	LoRA estándar	k	3, 6, 9, 12	4	24,576	62.5	0.686	−0.019
6 s	LoRA progresivo	v	12	16	24,576	77.1	0.811	+0.106
6 s	LoRA inteligente	v y o	10–12	16	147,456	79.2	0.835	+0.130
7.5 s	LoRA estándar	k	3, 6, 9, 12	4	24,576	72.9	0.668	−0.087
7.5 s	LoRA progresivo	q y k	8–12	4	86,016	77.1	0.747	−0.008
7.5 s	LoRA inteligente	v y o	8–9	4	24,576	77.1	0.818	+0.063

Este último caso es el más revelador. Adaptando las matrices q y k, LoRA progresivo alcanza la misma exactitud que LoRA inteligente (77.1 %), pero su AUC (0.747) es inferior al del modelo congelado (0.755). Es decir, acierta en el umbral elegido, pero no ordena mejor a pacientes y controles que el baseline. LoRA inteligente, con las matrices v y o, alcanza la misma exactitud con un AUC de 0.818.

La consistencia de este patrón es relevante porque se mantiene aunque cambien casi todas las demás condiciones del experimento. Entre ventanas cambia el bloque de capas seleccionado (C en 6 s, B en 7.5 s), el rango de LoRA (16 en 6 s, 4 en 7.5 s) y el número de parámetros entrenados (147,456 frente a 24,576). Lo único que se conserva en la configuración inteligente son las matrices v y o, y en ambos casos producen la mejor capacidad de discriminación de la familia.

Una interpretación de este resultado surge del papel de cada matriz en el mecanismo de atención. Las matrices q y k determinan hacia dónde atiende cada instante de la señal, es decir, qué fragmentos del audio se relacionan entre sí. Las matrices v y o determinan qué información se extrae de esos fragmentos y cómo se incorpora a la representación que pasa a la siguiente capa. Los resultados sugieren que los patrones de atención aprendidos durante el preentrenamiento ya son adecuados para capturar la estructura temporal del habla, y que lo que requiere ajuste es el contenido de la representación, la información acústica y prosódica relevante para la AD, que el ajuste previo del modelo para reconocimiento de voz tiende a atenuar.

+VI. CONCLUSIONES Y RECOMENDACIONES

Los resultados de este trabajo permiten afirmar que el ajuste eficiente de parámetros es una alternativa viable y preferible al ajuste de pesos del modelo para detectar AD a partir del habla cuando los datos son escasos. Sin embargo, esta ventaja no es automática; depende de decisiones de diseño que suelen tratarse como secundarias.

La primera de estas decisiones es la duración del audio que recibe el modelo. wav2vec 2.0 construye la representación de cada instante a partir del contexto que lo rodea, y fue preentrenado con fragmentos largos. Cuando se le entregan fragmentos cortos, pierde ~~precisamente~~ el contexto del que dependen sus representaciones. Antes de ajustar el modelo, ~~entonces~~, es necesario garantizar que cada fragmento le ofrezca un contexto comparable al de su preentrenamiento.

La segunda, ~~y la mas importante~~, es la ubicación de la adaptación. El número de parámetros entrenables no explica por sí solo el desempeño de los modelos. Se obtuvieron buenos resultados con configuraciones cuyo número de parámetros difiere en varios órdenes de magnitud, y resultados deficientes con presupuestos idénticos a los de configuraciones exitosas. Lo que tienen en común las configuraciones que superan al modelo congelado no es cuántos parámetros entrenan, sino dónde lo hacen, y una adaptación mal ubicada puede resultar incluso peor que no adaptar el modelo. La pregunta relevante al aplicar estos métodos es, en qué capas adaptar el modelo, y el análisis del contenido representacional de cada capa demostró ser una herramienta útil para responderla.

Por último, los resultados sugieren que el conocimiento que wav2vec 2.0 adquiere durante su preentrenamiento es transferible a esta tarea. La forma en que el modelo relaciona los distintos instantes de la señal a partir del contexto parece adecuada tal como está; lo que requiere ajuste es el contenido que el modelo extrae de ella. Los dos métodos con mejor desempeño apuntan en esa dirección. En LoRA, la adaptación más consistente fue la de las matrices asociadas al contenido de la atención. Los adaptadores bottleneck, por su parte, actúan como una traducción entre las representaciones de una capa y la siguiente, agregando una corrección sobre la información que el modelo ya produce. En ambos casos, la adaptación efectiva actúa sobre el contenido de las representaciones y no sobre la forma en que el modelo relaciona los distintos instantes de la señal.

En conjunto, este trabajo muestra que un modelo preentrenado para reconocimiento de voz puede adaptarse a la detección de AD sin modificar sus pesos originales, siempre que la adaptación se dirija a las capas y componentes cuyo contenido es relevante para la tarea.

REFERENCIAS

[1] “Global action plan on the public health response to dementia 2017 - 2025”. Consultado: el 25 de septiembre de 2026. [En línea]. Disponible en: https://www.who.int/publications/i/item/global-action-plan-on-the-public-health-response-to-dementia-2017---2025

[2] “La epidemia de demencia avanza silenciosa con el envejecimiento en Colombia”. Consultado: el 25 de septiembre de 2026. [En línea]. Disponible en: https://www.periodico.unal.edu.co/articulos/la-epidemia-de-demencia-avanza-silenciosa-con-el-envejecimiento-en-colombia

[3] A. Guerrero-Barragán y D. I. Lucumi-Cuesta, “Stakeholder perceptions of dementia in Colombia: a qualitative study”, *BMC Public Health*, vol. 25, núm. 1, p. 959, mar. 2025, doi: 10.1186/s12889-025-22166-9.

[4] A. Giudicessi *et al.*, “Cognitive Outcomes in Autosomal-Dominant Alzheimer’s Disease: A Comprehensive Review from a Colombian Kindred with the Presenilin-1 E280A Mutation”, *Journal of Alzheimer’s Disease*, vol. 101, núm. 2, pp. 397–415, sep. 2024, doi: 10.3233/JAD-240360.

[5] I. Vigo, L. Coelho, y S. Reis, “Speech- and Language-Based Classification of Alzheimer’s Disease: A Systematic Review”, *Bioengineering*, vol. 9, núm. 1, p. 27, ene. 2022, doi: 10.3390/bioengineering9010027.

[6] A. D. Assfaw, S. E. Schindler, y J. C. Morris, “Advances in blood biomarkers for Alzheimer disease (AD): A review”, *The Kaohsiung J of Med Scie*, vol. 40, núm. 8, pp. 692–698, ago. 2024, doi: 10.1002/kjm2.12870.

[7] A. Baevski, H. Zhou, A. Mohamed, y M. Auli, “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations”, el 22 de octubre de 2020, *arXiv*: arXiv:2006.11477. doi: 10.48550/arXiv.2006.11477.

[8] S. Luz, F. Haider, S. de la Fuente, D. Fromm, y B. MacWhinney, “Alzheimer’s Dementia Recognition through Spontaneous Speech: The ADReSS Challenge”, el 5 de agosto de 2020, *arXiv*: arXiv:2004.06833. doi: 10.48550/arXiv.2004.06833.

[9] F. Haider, S. De La Fuente, y S. Luz, “An Assessment of Paralinguistic Acoustic Features for Detection of Alzheimer’s Dementia in Spontaneous Speech”, *IEEE J. Sel. Top. Signal Process.*, vol. 14, núm. 2, pp. 272–281, feb. 2020, doi: 10.1109/JSTSP.2019.2955022.

[10] C. D. Rios-Urrego, J. C. Vásquez-Correa, J. R. Orozco-Arroyave, y E. Nöth, “Transfer Learning to Detect Parkinson’s Disease from Speech In Different Languages Using Convolutional Neural Networks with Layer Freezing”, en *Text, Speech, and Dialogue*, vol. 12284, P. Sojka, I. Kopeček, K. Pala, y A. Horák, Eds., en Lecture Notes in Computer Science, vol. 12284. , Cham: Springer International Publishing, 2020, pp. 331–339. doi: 10.1007/978-3-030-58323-1_36.

[11] A. Pasad, J.-C. Chou, y K. Livescu, “Layer-wise Analysis of a Self-supervised Speech Representation Model”, el 3 de diciembre de 2022, *arXiv*: arXiv:2107.04734. doi: 10.48550/arXiv.2107.04734.

[12] V. Berisha, C. Krantsevich, G. Stegmann, S. Hahn, y J. Liss, “Are reported accuracies in the clinical speech machine learning literature overoptimistic?”, en *Interspeech 2022*, ISCA, sep. 2022, pp. 2453–2457. doi: 10.21437/Interspeech.2022-691.

[13] Y.-L. Liu, R. Feng, J.-H. Yuan, y Z.-H. Ling, “Clever Hans Effect Found in Automatic Detection of Alzheimer’s Disease through Speech”, en *Interspeech 2024*, sep. 2024, pp. 2435 –2439. doi: 10.21437/Interspeech.2024-1018.

[14] Y. Xiong, V. Berisha, J. Liss, y C. Chakrabarti, “Mitigating Overfitting During Speech Foundation Model Fine-tuning: Applications to Dysarthric Speech Detection”, en *Interspeech 2025*, ISCA, ago. 2025, pp. 2133–2137. doi: 10.21437/Interspeech.2025-567.

[15] J. Howard y S. Ruder, “Universal Language Model Fine-tuning for Text Classification”.

[16] N. Houlsby *et al.*, “Parameter-Efficient Transfer Learning for NLP”, el 13 de junio de 2019, *arXiv*: arXiv:1902.00751. doi: 10.48550/arXiv.1902.00751.

[17] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models”, el 16 de octubre de 2021, *arXiv*: arXiv:2106.09685. doi: 10.48550/arXiv.2106.09685.

[18] Z.-C. Chen, C.-L. Fu, C.-Y. Liu, S.-W. Li, y H. Lee, “Exploring Efficient-tuning Methods in Self-supervised Speech Models”, el 30 de enero de 2023, *arXiv*: arXiv:2210.06175. doi: 10.48550/arXiv.2210.06175.

[19] T. Purohit, B. Ruvolo, J. R. Orozco-Arroyave, y M. Magimai.-Doss, “Automatic Parkinson’s disease detection from speech: Layer selection vs adaptation of foundation models”, en *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Hyderabad, India: IEEE, abr. 2025, pp. 1–5. doi: 10.1109/ICASSP49660.2025.10890852.

[20] N. Houlsby *et al.*, “Parameter-Efficient Transfer Learning for NLP”, el 13 de junio de 2019, *arXiv*: arXiv:1902.00751. doi: 10.48550/arXiv.1902.00751.

[21] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models”, el 16 de octubre de 2021, *arXiv*: arXiv:2106.09685. doi: 10.48550/arXiv.2106.09685.

[22] D. S. Park *et al.*, “SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition”, en *Interspeech 2019*, ISCA, sep. 2019, pp. 2613–2617. doi: 10.21437/Interspeech.2019-2680.

[23] V. Lialin, V. Deshpande, X. Yao, y A. Rumshisky, “Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning”, el 22 de noviembre de 2024, *arXiv*: arXiv:2303.15647. doi: 10.48550/arXiv.2303.15647.

[24] J. Pfeiffer, A. Kamath, A. Rücklé, K. Cho, y I. Gurevych, “AdapterFusion: Non-Destructive Task Composition for Transfer Learning”, en *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, Online: Association for Computational Linguistics, 2021, pp. 487–503. doi: 10.18653/v1/2021.eacl-main.39.

[25] G. Strang, *Linear algebra and its applications*, 4th ed. Belmont, CA: Thomson, Brooks/Cole, 2006.

[26] “The Natural History of Alzheimer’s Disease: Description of Study Cohort and Accuracy of Diagnosis”.

[27] C. Wang *et al.*, “VoxPopuli: A Large-Scale Multilingual Speech Corpus for Representation Learning, Semi-Supervised Learning and Interpretation”, en *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Online: Association for Computational Linguistics, 2021, pp. 993–1003. doi: 10.18653/v1/2021.acl-long.80.

ANEXOS

En los anexos se incluye material complementario que apoya la documentación investigativa, tales como consentimientos informados, entrevistas, material fotográfico, etc. Evite incluir material que puede estar protegido por derechos de autor, tales como pruebas psicológicas, fragmentos de libros, artículos de revistas, patentes, etc. Recuerda no incluir en tu documento datos de personas o entidades objetos de la investigación, tales como nombres, apellidos, cédulas, números telefónicos, consentimientos informados con datos personales (Resolución 8430 de 1993), nombres de empresas sin el consentimiento escrito del representante legal, fotografías en primer plano de personas (especialmente de menores de edad) y demás información que pueda contravenir los principios emitidos en la Ley Estatutaria 1581 de 2012 (Ley de protección de datos personales).

Los siguientes anexos contienen documentos de interés para el proceso de trabajo de grado, así como trucos y recomendaciones que surgen constantemente en la elaboración de un documento en Word.

Anexo A. Autoarchivo en Repositorio y documentos de interés

Consulte el instructivo de autoarchivo en el este enlace.

Anexo B. Póster Autoarchivo en Repositorio y documentos de interés

En último lugar, se incluye el anexo del poster que utilizarás en la jornada de prácticas académicas, en caso de aplicar. Guárdelo de forma tal que no modifique el tamaño de las hojas ni afecte la presentación del documento.